Deep Gaussian Processes

Adam Buckley

Lancaster University

adam@brownthwaite.com

Abstract

Gaussian processes (GPs) provide a non-parametric framework for Bayesian inference, providing predictions with principled and calibrated uncertainty estimates. The expressiveness of a single GP is limited by its kernel, which encodes a fixed notion of smoothness across the entire input space. Deep Gaussian Processes (DGPs) aim to address this limitation by composing multiple GP layers into a hierarchy, allowing the model to learn representations at multiple levels of abstraction while retaining the Bayesian uncertainty quantification of standard GPs. This expressiveness comes at a cost of intractable inference, which leads DGPs to require approximate methods for training.

This dissertation presents the theory of GPs and DGPs, derives the variational inference framework needed to make DGP training tractable at scale, and evaluates both model families on two benchmark datasets. We begin from the weight-space view of Bayesian linear regression and build to the function-space perspective of GPs, then show how composing GP layers leads to a non-Gaussian joint distribution and motivates doubly stochastic variational inference.

Two experiments are carried out: a regression benchmark on the Kin8nm robot kinematics dataset and a binary classification benchmark on the UCI credit card default dataset. On Kin8nm, DSVGP-2L achieves a 10.7% RMSE reduction over the single-layer GP, suggesting that depth adds value on non-stationary, compositional targets. On the credit card task, the GP family matches XGBoost on discrimination (ROC-AUC within ± 0.003) while delivering 4–22$\times$ better native probability calibration, an important property for regulatory probability-of-default estimation. In both experiments, three-layer DGPs never meaningfully outperform two-layer variants and degrade badly at small data scales.

1.0 Introduction

Many real-world prediction tasks require not only accurate point estimates, but also reliable uncertainty. A model that can communicate how confident it is, and where it should not be trusted, is often more useful than one that cannot. Gaussian processes (GPs) provide a principled framework for this (Rasmussen and Williams 2006). GPs are Bayesian non-parametric models, and maintain a distribution over functions directly, giving natively calibrated predictive distributions without the need for post-hoc calibration (Bishop 2006). This gives GPs an advantage in several aspects over classical machine learning models, which often require additional methods to produce calibrated uncertainty estimates.

However, a single GP is limited by its kernel, which encodes a fixed global prior over function smoothness. This prevents a GP from adapting to target functions with varying local structure or multi-scale behaviour, which are common in real-world data (Duvenaud et al. 2014).

Deep Gaussian Processes (DGPs) (Damianou and Lawrence 2013) address this limitation by composing multiple GP layers into a hierarchy. Each layer maps its inputs into a learned latent representation that is passed to the next, allowing the model to adapt its effective input space before the final prediction layer. However, the composition produces a non-Gaussian joint distribution, making exact inference intractable and requiring approximate methods for training.

This dissertation begins by introducing the theory of GPs, starting from the weight-space view of Bayesian linear regression and building to the function-space perspective of GPs. We then show how composing GP layers into DGPs leads to a non-Gaussian joint distribution and motivates doubly stochastic variational inference for training DGPs at scale (Salimbeni and Deisenroth 2017). We finish by evaluating GPs and DGPs against classical baselines on both a regression and a classification benchmark.

2.0 Towards Gaussian Processes

In regression we observe a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ of inputs $\mathbf{x}_i \in \mathbb{R}^D$ and real-valued outputs $y_i \in \mathbb{R}$, and want to learn a function $f : \mathbb{R}^D \to \mathbb{R}$ such that $y = f(\mathbf{x}) + \epsilon$, where ϵ is noise.

In a classical frequentist approach, we typically define the function f as a parametric model and estimate it by using a finite set of parameters $\mathbf{w}$. For example, in linear regression we have $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$, and we aim to find the best estimate $\hat{\mathbf{w}}$ such that $y_i \approx \hat{\mathbf{w}}^\top \mathbf{x}_i$ for all i. In Bayesian parametric models, we place a prior distribution

over the parameters $\mathbf{w}$ and compute a posterior distribution $p(\mathbf{w} \mid \mathcal{D})$ that captures updated beliefs about the parameters after observing the data (Bishop 2006; Murphy 2012).

In these parametric models, the function f is fully specified by a finite set of parameters $\mathbf{w}$. This number and structure of parameters are fixed in advance, and therefore the function space of f is limited by the choice of model. Additionally, the model class and its parametrisation must be chosen before observing the data, which requires domain expertise. Only the parameter values are then learned from the data.

In contrast to parametric approaches that specify f via a finite-dimensional model, GPs place a prior directly on an infinite-dimensional function space (Rasmussen and Williams 2006). Observed data then reduces uncertainty, leaving a posterior distribution that captures updated beliefs about the function.

This section outlines the theory of GPs for regression tasks, starting from the weight-space view of Bayesian linear regression and building up to the function-space view of GPs.

2.1 The Weight Space View

The weight-space view treats regression as Bayesian inference over a weight vector (Rasmussen and Williams 2006; MacKay 1998). To allow non-linear functions, we replace the raw inputs in $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{w}$ with a set of basis functions:

$$f(\mathbf{x}) = \sum_{j=1}^{M} w_j \phi_j(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x})^\top \mathbf{w},$$

where $\boldsymbol{\phi}(\mathbf{x}) = [\phi_1(\mathbf{x}), \ldots, \phi_M(\mathbf{x})]^\top$ is a vector of basis functions, which maps the input to an M-dimensional feature space, with weights $\mathbf{w} = [w_1, \ldots, w_M]^\top$. This becomes more general than the simple linear model, allowing the modeling of non-linear relationships through the choice of basis functions $\phi_j(\cdot)$, without breaking linearity in the parameters $\mathbf{w}$.

Given some training data $\mathcal{D}$, our goal is to learn a set of weights $\mathbf{w}$ that best describes the data, $\hat{\mathbf{w}}$. A common approach is to use maximum likelihood estimation (MLE) (Bishop 2006; Murphy 2012), which finds the single best set of weights that maximises the likelihood of the observed data. However, this process ignores how it is possible for different weight vectors to explain the observed data equally well, while making very different predictions in unobserved regions. MLE can also lead to overfitting, where the model fits the training data too closely and fails to generalise to new data.

2.2 Bayesian Linear Regression

Rather than committing to a single $\hat{\mathbf{w}}$, *Bayesian linear regression* treats the weights as uncertain: we place a prior over $\mathbf{w}$ and update it with the data to obtain a posterior, which captures our uncertainty about the weights and therefore about the function itself (Bishop 2006; Gelman et al. 2013).

2.2.1 Prior and Likelihood

A natural choice of prior is a zero-mean Gaussian:

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \Sigma_p),$$

where $\Sigma_p \in \mathbb{R}^{M \times M}$ is the prior covariance matrix. A common prior is $\Sigma_p = \sigma_w^2 \mathbf{I}$, which gives independent, identically distributed, (i.i.d.) Gaussian weights with variance σ_w^2 (MacKay 1992).

With independent Gaussian noise $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$, each observation $y_i = f(\mathbf{x}_i) + \epsilon_i$ has likelihood

$$p(y_i \mid \mathbf{x}_i, \mathbf{w}) = \mathcal{N}(y_i \mid \boldsymbol{\phi}(\mathbf{x}_i)^\top \mathbf{w}, \sigma_n^2).$$

Since observations are independent given $\mathbf{w}$, the joint likelihood becomes the product of the individual likelihoods:

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \prod_{i=1}^{n} \mathcal{N}(y_i \mid \boldsymbol{\phi}(\mathbf{x}_i)^\top \mathbf{w}, \sigma_n^2). \tag{1}$$

Collecting observations into the *design matrix* $\Phi \in \mathbb{R}^{n \times M}$ with $\Phi_{ij} = \phi_j(\mathbf{x}_i)$ (Rasmussen and Williams 2006) (see Appendix 1.1 for the explicit construction), Equation 1 becomes a single multivariate Gaussian rather than a product of univariate Gaussians:

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \mathcal{N}(\mathbf{y} \mid \Phi\mathbf{w}, \sigma_n^2 \mathbf{I}).$$

2.2.2 Posterior over Weights

Applying Bayes' theorem to this gives the posterior distribution over weights:

$$p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \frac{p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) p(\mathbf{w})}{p(\mathbf{y} \mid \mathbf{X})}.$$

Because the prior and likelihood are both Gaussian, their product is proportional to another Gaussian, so the posterior is also Gaussian (Bishop 2006; Bernardo and

Smith 2000). A prior that yields a posterior in its own family is called *conjugate* to the likelihood (Gelman et al. 2013), and conjugacy is what allows closed-form inference.

Evaluating the posterior gives our covariance:

$$\Sigma_w = \left(\sigma_n^{-2}\Phi^\top\Phi + \Sigma_p^{-1}\right)^{-1},$$

and posterior mean:

$$\bar{\mathbf{w}} = \sigma_n^{-2}\Sigma_w\Phi^\top\mathbf{y}. \tag{2}$$

The posterior distribution over the weights is therefore:

$$p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \mathcal{N}(\mathbf{w} \mid \bar{\mathbf{w}}, \Sigma_w).$$

See Appendix 1.2 for the full derivation of the posterior mean and covariance.

2.3 Predictive Distribution

This posterior distribution captures both the best estimate of the weights $\bar{\mathbf{w}}$, and the uncertainty about the weights Σ_w. We can use this to make predictions at a new input $\mathbf{x}_*$ by marginalising over the weight posterior. Here we predict the noisy observation $y_* = f(\mathbf{x}_*) + \epsilon_*$ where $\epsilon_* \sim \mathcal{N}(0, \sigma_n^2)$:

$$p(y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}) = \int p(y_* \mid \mathbf{x}_*, \mathbf{w}) p(\mathbf{w} \mid \mathbf{X}, \mathbf{y})\, d\mathbf{w}, \tag{3}$$

where $p(y_* \mid \mathbf{x}_*, \mathbf{w}) = \mathcal{N}(y_* \mid \boldsymbol{\phi}(\mathbf{x}_*)^\top\mathbf{w}, \sigma_n^2)$. Since $f(\mathbf{x}_*) = \boldsymbol{\phi}(\mathbf{x}_*)^\top\mathbf{w}$ is a linear function of $\mathbf{w}$, the marginal $p(y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y})$ is also Gaussian (Bishop 2006, chap. 3). Evaluating the integral in closed form gives the predictive mean and variance (see Appendix 1.3 for the full derivation):

$$\begin{aligned} \text{mean:} \quad & \mathbb{E}[y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}] = \boldsymbol{\phi}(\mathbf{x}_*)^\top\bar{\mathbf{w}}, \\ \text{variance:} \quad & \mathrm{Var}[y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}] = \boldsymbol{\phi}(\mathbf{x}_*)^\top\Sigma_w\boldsymbol{\phi}(\mathbf{x}_*) + \sigma_n^2. \end{aligned} \tag{4}$$

2.4 From Weight Space to Function Space

The predictive distribution in Equation 4 is expressed in terms of the feature map $\boldsymbol{\phi}$. However, we can see $\boldsymbol{\phi}$ appears only through inner products weighted by the prior covariance Σ_p, of the form $\boldsymbol{\phi}(\mathbf{x})^\top \Sigma_p \boldsymbol{\phi}(\mathbf{x}')$. We can define a *kernel function* (interchangeably called a *covariance function*) as an inner product in the feature space: (Schölkopf and Smola 2002; Rasmussen and Williams 2006)

$$k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^\top \Sigma_p \boldsymbol{\phi}(\mathbf{x}'),$$

which measures similarity between inputs in the feature space, weighted by the prior covariance Σ_p.

With this definition, we can collect all pairwise kernel evaluations over the full training set into the *kernel matrix* $K \in \mathbb{R}^{n \times n}$ with $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$, and the vector of cross-covariances $\mathbf{k}_* \in \mathbb{R}^n$ with $[\mathbf{k}_*]_i = k(\mathbf{x}_i, \mathbf{x}_*)$. This allows for rewriting the predictive equation entirely in terms of the kernel. This can be done by reparametrising the posterior covariance $\Sigma_w = (\sigma_n^{-2}\Phi^\top\Phi + \Sigma_p^{-1})^{-1}$ using the matrix inversion lemma (Rasmussen and Williams 2006, Equation A.9),

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U\Big(C^{-1} + VA^{-1}U\Big)^{-1}VA^{-1},$$

setting $A = \Sigma_p^{-1}$, $U = \Phi^\top$, $C = \sigma_n^{-2}\mathbf{I}$, $V = \Phi$, which gives the equivalent $n \times n$ expression:

$$\Sigma_w = \Sigma_p - \Sigma_p\Phi^\top\Big(\sigma_n^2\mathbf{I} + \Phi\Sigma_p\Phi^\top\Big)^{-1}\Phi\Sigma_p. \tag{5}$$

In matrix form $K = \Phi\Sigma_p\Phi^\top$ and $\mathbf{k}_* = \Phi\Sigma_p\boldsymbol{\phi}(\mathbf{x}_*)$. Substituting Equation 5 into the predictive mean $\mathbb{E}[y_*] = \sigma_n^{-2}\boldsymbol{\phi}(\mathbf{x}_*)^\top\Sigma_w\Phi^\top\mathbf{y}$:

$$\begin{aligned}
\mathbb{E}[y_*] &= \sigma_n^{-2}\boldsymbol{\phi}(\mathbf{x}_*)^\top\Sigma_p\Phi^\top\mathbf{y} - \sigma_n^{-2}\mathbf{k}_*^\top(K + \sigma_n^2\mathbf{I})^{-1}K\mathbf{y} \\
&= \mathbf{k}_*^\top\Big[\sigma_n^{-2}\mathbf{I} - \sigma_n^{-2}(K + \sigma_n^2\mathbf{I})^{-1}K\Big]\mathbf{y} \\
&= \mathbf{k}_*^\top(K + \sigma_n^2\mathbf{I})^{-1}\mathbf{y},
\end{aligned}$$

where the last step uses $\sigma_n^{-2}(K + \sigma_n^2\mathbf{I})^{-1}[(K + \sigma_n^2\mathbf{I}) - K] = (K + \sigma_n^2\mathbf{I})^{-1}$. Substituting into the variance $\mathrm{Var}[y_*] = \boldsymbol{\phi}(\mathbf{x}_*)^\top\Sigma_w\boldsymbol{\phi}(\mathbf{x}_*) + \sigma_n^2$:

$$\begin{aligned}
\mathrm{Var}[y_*] &= \boldsymbol{\phi}(\mathbf{x}_*)^\top\Sigma_p\boldsymbol{\phi}(\mathbf{x}_*) - \mathbf{k}_*^\top(K + \sigma_n^2\mathbf{I})^{-1}\mathbf{k}_* + \sigma_n^2 \\
&= k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top(K + \sigma_n^2\mathbf{I})^{-1}\mathbf{k}_* + \sigma_n^2.
\end{aligned}$$

The predictive distribution in terms of the kernel is therefore:

$$\begin{aligned}\mathbb{E}[y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}] &= \mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}, \\ \mathrm{Var}[y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}] &= k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_* + \sigma_n^2.\end{aligned} \tag{6}$$

Substituting these into the Gaussian predictive distribution from Equation 3 gives:

$$p(y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}) = \mathcal{N}\Big(y_* \;\Big|\; \mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y},\; k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_* + \sigma_n^2\Big)\,. \tag{7}$$

The most important observation here is that $\boldsymbol{\phi}$ has vanished entirely. The predictions depend purely on kernel evaluations between data points. We are free to specify k directly, and we never need to construct an explicit feature map. If we allow $M \to \infty$ many basis functions, and we define k to be the corresponding inner product of this limit, then the model becomes *non-parametric*. The function's complexity grows with the data rather than being fixed in advance (Williams 1996; MacKay 1998). Building on the work of Neal (1996), Williams (1996) showed a Bayesian neural network with infinite hidden units and a Gaussian weight prior converges to a GP, proving the connection between weight-space and function-space views.

This is the *function-space perspective*. Instead of reasoning about a weight vector $\mathbf{w}$, we place a prior *directly over functions*, characterised entirely by k. The result is a Gaussian process.

3.0 Gaussian Processes

Formally, a Gaussian process is a collection of random variables, any finite number of which have a joint Gaussian distribution (Rasmussen and Williams 2006). A GP is fully specified by its mean function $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$ and covariance function (or kernel)

$$k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - m(\mathbf{x}))(f(\mathbf{x}') - m(\mathbf{x}'))] = \mathrm{Cov}[f(\mathbf{x}), f(\mathbf{x}')]$$

We write $f \sim \mathcal{GP}(m, k)$ to denote that f is a random function drawn from a GP with mean m and kernel k. The GP provides a prior directly over functions, rather than over a finite parameter vector as in the weight-space view of Section 2.4.

3.1 The GP Prior

Placing a GP prior on f gives a "prior over functions". Evaluating f at any finite collection of inputs $\mathbf{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ gives a joint Gaussian over the corresponding function values $\mathbf{f} = [f(\mathbf{x}_1), \ldots, f(\mathbf{x}_n)]^\top$,

$$\mathbf{f} \mid \mathbf{X} \sim \mathcal{N}(\mathbf{m}, K),$$

where $[\mathbf{m}]_i = m(\mathbf{x}_i)$ and $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$. This property that every finite-dimensional marginal is Gaussian is a defining characteristic of the process (Rasmussen and Williams 2006; Williams and Rasmussen 1995). In regression, it is standard practice to take $m \equiv 0$: the training targets can be centred by subtracting their mean, and any residual structure is captured by the kernel (MacKay 1998). However, non-zero mean functions can be useful for encoding strong prior beliefs about the function shape, such as a linear or polynomial trend (Rasmussen and Williams 2006, sec. 2.2).

3.2 Kernels and Covariance Functions

The structure of the prior is fully encoded in the choice of kernel k. The kernel determines how function values co-vary across input space: inputs that are 'close', as measured by the kernel k, will have strongly correlated function values, while distant inputs are nearly independent. Kernel hyperparameters control interpretable properties such as the lengthscale (how rapidly the function varies) and the output scale (the typical amplitude), while the choice of kernel itself determines structural properties such as stationarity or periodicity. Figure 1 illustrates this with two common choices: the squared exponential (SE) kernel, which produces infinitely differentiable functions whose roughness is controlled by a lengthscale parameter ℓ, and the periodic kernel, whose period parameter p fixes the repetition frequency of the sampled functions.

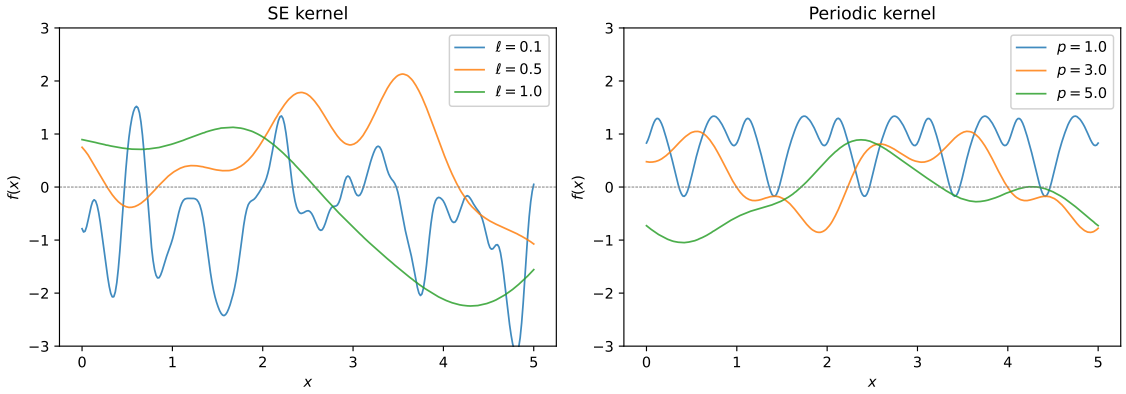


Figure 1: Prior samples from GPs with the SE kernel (left) and the periodic kernel (right) under varying hyperparameters.

The SE kernel, which is interchangeably called the radial basis function (RBF) kernel, is defined as

$$k_{\text{SE}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right),$$

and is the most widely used GP kernel in practice (Rasmussen and Williams 2006). A small ℓ allows the function to vary rapidly over short distances, producing rough wiggly samples, while a large ℓ enforces smoothly varying functions. The periodic kernel,

$$k_{\text{per}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{2\sin^2(\pi|\mathbf{x} - \mathbf{x}'|/p)}{\ell^2}\right),$$

explicitly encodes a repeating structure with period p, making it suitable for modelling seasonal or cyclical signals.

Other common kernels include the Matérn family, which has an additional smoothness parameter that controls the differentiability of the sampled functions, and the rational quadratic kernel, which can be seen as a scale mixture of SE kernels and can capture varying lengthscales (Rasmussen and Williams 2006, secs. 4.2–4.3).

Kernels can also be combined: both sums and products of valid kernels are themselves valid kernels. This enables the construction of structured priors that capture multiple scales or mixed behaviours (Schölkopf and Smola 2002).

3.3 Choosing Kernel Hyperparameters

The model hyperparameters $\boldsymbol{\theta}$ (e.g. kernel lengthscale ℓ and output scale σ_f^2, and likelihood noise variance σ_n^2) are not specified by the GP directly, and so must

be estimated from data. The standard approach is to maximise the log marginal likelihood (LML) of the observed data under the GP model, which is known as *type-II* or *level-II* maximum likelihood or sometimes as *empirical Bayes* (MacKay 1992; MacKay 1998),

$$\begin{aligned}\log p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\theta}) = &-\frac{1}{2}\mathbf{y}^\top (K + \sigma_n^2 \mathbf{I})^{-1}\mathbf{y} \quad \text{(data fit)} \\ &-\frac{1}{2}\log |K + \sigma_n^2 \mathbf{I}| \quad \text{(complexity penalty)} \\ &-\frac{n}{2}\log 2\pi \quad \text{(normalisation constant)}.\end{aligned} \tag{8}$$

This LML has three interpretable terms: the first encourages the model to fit the data well, the second penalises overly complex models that can fit many functions, and the third is a constant that ensures the likelihood is properly normalised. The LML is typically maximised using gradient-based optimisation (e.g. L-BFGS or Adam (Kingma and Ba 2015)), which is tractable for exact GPs since the LML and its gradients can be computed in $\mathcal{O}(n^3)$ time via Cholesky decomposition of $K + \sigma_n^2 \mathbf{I}$. However, the LML is non-convex in general, and in low-dimensional settings can exhibit multiple local optima; it is common to use multiple random restarts or a good initialisation strategy to mitigate this (Rasmussen and Williams 2006). Additionally, since hyperparameters such as lengthscale and noise variance must be positive, it is standard practice to optimise them in log-space to ensure positivity and improve numerical stability.

3.4 GP Regression

Given noisy training observations $\mathbf{y} = \mathbf{f} + \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_n^2 \mathbf{I})$ independent of $\mathbf{f}$, the three blocks of the joint covariance follow directly from the GP prior (see Appendix 1.4 for the full expansion):

$$\begin{aligned}\operatorname{Cov}[\mathbf{y}, \mathbf{y}] &= \operatorname{Cov}[\mathbf{f} + \boldsymbol{\epsilon}, \mathbf{f} + \boldsymbol{\epsilon}] = K + \sigma_n^2 \mathbf{I}, \\ \operatorname{Cov}[\mathbf{y}, f_*] &= \operatorname{Cov}[\mathbf{f} + \boldsymbol{\epsilon}, f(\mathbf{x}_*)] = \mathbf{k}_*, \\ \operatorname{Cov}[f_*, f_*] &= k(\mathbf{x}_*, \mathbf{x}_*) = k_{**},\end{aligned}$$

where $[\mathbf{k}_*]_i = k(\mathbf{x}_i, \mathbf{x}_*)$ and $k_{**} = k(\mathbf{x}_*, \mathbf{x}_*)$. Since $\mathbf{y}$ and f_* are jointly Gaussian as linear functions of the GP, the joint distribution over training outputs and test value $f_* = f(\mathbf{x}_*)$ is:

$$\begin{bmatrix}\mathbf{y} \\ f_*\end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix}K + \sigma_n^2 \mathbf{I} & \mathbf{k}_* \\ \mathbf{k}_*^\top & k_{**}\end{bmatrix}\right).$$

Conditioning on $\mathbf{y}$ via the standard Gaussian conditionals (Rasmussen and Williams 2006, Equation 2.19; Bishop 2006, sec. 2.3) gives the posterior predictive distribution:

$$p(y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}) = \mathcal{N}\Big(y_* \;\Big|\; \mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y},\; k_{**} - \mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_* + \sigma_n^2\Big). \tag{9}$$

This is identical to the weight-space result in Equation 7, but expressed in terms of the kernel rather than the feature map. The posterior mean and covariance are therefore the same as the predictive mean and variance from Equation 6, and we can write the posterior GP as:

$$\begin{aligned} m_{\text{post}}(\mathbf{x}_*) &= \mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}, \\ k_{\text{post}}(\mathbf{x}_*, \mathbf{x}_*) &= k_{**} - \mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_*. \end{aligned} \tag{10}$$

These expressions have intuitive interpretations: the posterior mean is a linear combination of the observed outputs $\mathbf{y}$, where the weights are determined by the similarity between the test input and training inputs, as measured by the kernel. The posterior variance $k_{\text{post}}(\mathbf{x}_*, \mathbf{x}_*)$ is the prior variance at $\mathbf{x}_*$ minus a positive term that captures how much information about $f(\mathbf{x}_*)$ is contained in the training data. As more data are observed near $\mathbf{x}_*$, this reduction grows and uncertainty falls. Specifically, $\mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_*$ is large when $\mathbf{x}_*$ is strongly correlated with the training inputs, with $(K + \sigma_n^2 \mathbf{I})^{-1}$ penalising duplicate, redundant or noisy observations. The full predictive distribution for a new *noisy* observation $y_* = f(\mathbf{x}_*) + \epsilon_*$ adds $+\sigma_n^2$ to the latent posterior variance, giving Equation 9.

Figure 2 illustrates these properties: the left panel shows five samples from the GP prior, and the right panel shows the posterior after conditioning on observations, where uncertainty collapses near the data and the posterior mean tracks the observations.

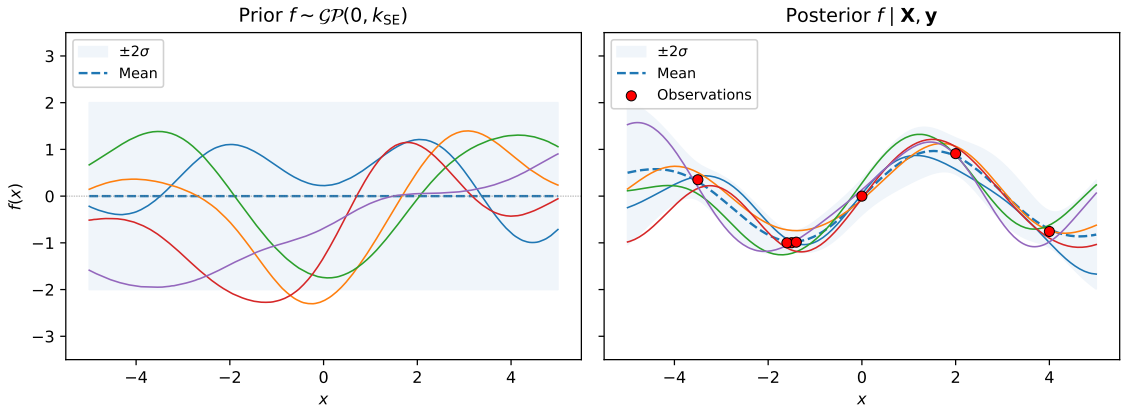


Figure 2: GP regression with an SE kernel. Left: five prior samples before observing data. Right: posterior mean (blue), 95% credible interval (shaded), and five posterior samples after conditioning on seven observations (red dots).

3.5 Training a GP

Each evaluation of the LML in Equation 8 and its gradients requires an $\mathcal{O}(n^3)$ Cholesky decomposition of $K + \sigma_n^2 \mathbf{I}$. This is tractable for moderate n but prohibitive for large datasets, which motivates the sparse approximations of Section 5.0.

3.5.1 Gradient of the Log Marginal Likelihood

Using the matrix derivative trace formula (Rasmussen and Williams 2006, Equation 5.9), the gradient of Equation 8 with respect to a hyperparameter θ_j is:

$$\frac{\partial \log p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\theta})}{\partial \theta_j} = \frac{1}{2}\mathbf{y}^\top (K + \sigma_n^2 \mathbf{I})^{-1} \frac{\partial (K + \sigma_n^2 \mathbf{I})}{\partial \theta_j} (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y} - \frac{1}{2} \mathrm{tr}\left((K + \sigma_n^2 \mathbf{I})^{-1} \frac{\partial (K + \sigma_n^2 \mathbf{I})}{\partial \theta_j} \right).$$

Defining $K_\alpha = K + \sigma_n^2 \mathbf{I}$ and $\boldsymbol{\alpha} = K_\alpha^{-1} \mathbf{y}$, the gradient has two interpretable terms. The trace $\mathrm{tr}(A)$ is the sum of diagonal entries of a matrix A; it appears here via the identity $\frac{\partial \log |A|}{\partial \theta_j} = \mathrm{tr}\left(A^{-1} \frac{\partial A}{\partial \theta_j}\right)$, so the second term is the negative gradient of the log-determinant $\log |K_\alpha|$. The first term, $\frac{1}{2} \boldsymbol{\alpha}^\top \frac{\partial K_\alpha}{\partial \theta_j} \boldsymbol{\alpha}$, is the data-fit gradient which is large when the change in kernel structure aligns with the pattern in the posterior weights $\boldsymbol{\alpha}$. This term therefore rewards hyperparameters whose covariance better explains $\mathbf{y}$. The second term, $-\frac{1}{2} \mathrm{tr}(K_\alpha^{-1} \frac{\partial K_\alpha}{\partial \theta_j})$, penalises growth in the kernel's

effective degrees of freedom regardless of the data. So the total result is a balance of these terms where hyperparameters increase only as much as the improvement in the data fit justifies the added complexity.

In practice, the training loop iterates: (1) evaluate the negative LML via Cholesky factorisation of $K_\theta + \sigma_n^2 \mathbf{I}$; (2) backpropagate to obtain $\partial \mathcal{L} / \partial \theta_j$ for each hyperparameter; (3) apply an update step (e.g. Adam). Hyperparameters are optimised in log-space, as noted in Section 3.3.

Figure 3 illustrates the effect of training: a GP with a poorly initialised lengthscale produces an oversmoothed, miscalibrated fit, while the same model after 50 Adam steps accurately recovers the latent function.

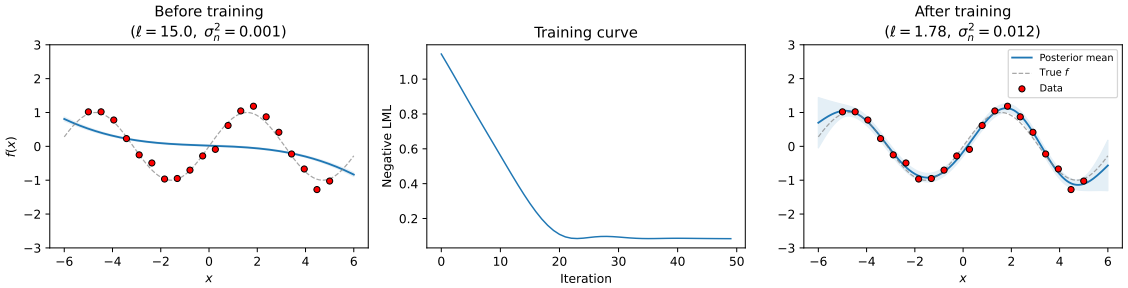


Figure 3: Effect of hyperparameter training on GP fit. Left: GP with an intentionally large initial lengthscale and a near-zero noise variance, giving a flat oversmoothed posterior. Centre: negative log marginal likelihood during 50 Adam training iterations. Right: GP after training. The learnt hyperparameter values are shown above the plots.

3.6 Prediction

After learning, with hyperparameters now fixed at their optimised values $\hat{\boldsymbol{\theta}}$, the task of predicting a new observation y_* at a test input $\mathbf{x}_*$ reduces to evaluating the posterior from Equation 10 at $\mathbf{x}_*$. The main cost for a prediction is the Cholesky factorisation $LL^\top = K_{\hat{\boldsymbol{\theta}}} + \sigma_n^2 \mathbf{I}$, which is $\mathcal{O}(n^3)$ and is only performed once. Once cached, the predictive mean at a new point is a single dot product with the precomputed vector $\boldsymbol{\alpha} = (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}$, which has $\mathcal{O}(n)$ complexity. The predictive variance additionally requires $\mathcal{O}(n^2)$ per test point. The full predictive distribution is therefore $\mathcal{O}(n^2)$ per test point. Prediction at test time is still much cheaper than the $\mathcal{O}(n^3)$ training cost, and the main challenge is the training phase when dealing with large datasets.

The predictive variance at a test point $\mathbf{x}_*$ can be split into two parts:

$$\mathrm{Var}[y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}, \hat{\boldsymbol{\theta}}] = \underbrace{k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (K + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_*}_{\text{epistemic (reducible)}} + \underbrace{\sigma_n^2}_{\text{aleatoric (irreducible)}},$$

where the first term is the *epistemic* uncertainty which is the model's ignorance about f. The second is the *aleatoric* noise (Kendall and Gal 2017). The epistemic term decreases as training data accumulates near $\mathbf{x}_*$, and equals the full prior variance $k(\mathbf{x}_*, \mathbf{x}_*)$ where no data has been observed. The σ_n^2 term is always present when predicting a noisy observation y_*, but absent when predicting the latent $f(\mathbf{x}_*)$ directly. It is an irreducible hard floor on prediction uncertainty that no amount of data can eliminate.

This is visible in Figure 2: uncertainty collapses near the training observations (low epistemic uncertainty) and inflates in the gaps and at the extrapolation boundaries (high epistemic uncertainty), while the posterior mean threads through the data.

4.0 Deep Gaussian Processes

The previous section shows how GPs provide closed-form predictions and uncertainty estimates, and are entirely described by the choice of kernel. However, the expressiveness of a single-layer GP is fundamentally constrained by this. A stationary kernel like the squared exponential encodes a fixed idea of smoothness across the entire input space, which may not match real-world data where patterns and smoothness of data may vary in complexity across different regions.

Deep Gaussian Processes (DGPs) address this by composing multiple GP layers hierarchically. Rather than relying on a single kernel, DGPs learn representations at multiple levels of abstraction, much like deep neural networks, while retaining Bayesian uncertainty quantification. This section formally describes the DGP model and shows why exact inference becomes intractable. This leads into the approximate methods of Section 5.0.

4.1 Definition and Model Formulation

A DGP composes multiple GP layers in a hierarchical structure (Damianou and Lawrence 2013). Instead of a single mapping from inputs to outputs, each layer defines a latent function that maps its inputs into a new latent representation, which then serves as the input to the next layer.

An L-layer DGP is defined by a sequence of latent functions $\{f^{(\ell)}\}_{\ell=1}^{L}$ where each function is drawn from a GP prior conditioned on the output of the previous:

$$f^{(\ell)} \mid \mathbf{F}^{(\ell-1)} \sim \mathcal{GP}\left(0, k^{(\ell)}(\cdot,\cdot)\right), \quad \ell = 1, \ldots, L,$$

with the convention $\mathbf{F}^{(0)} = \mathbf{X}$ (the observed inputs). Each layer applies a freshly drawn GP function to the *outputs* of the previous layer, where the first layer warps the raw data inputs, the second warps those warped values, and so on. The model is a learned chain of transformations, with each layer inheriting all the uncertainty of the layers before it.

For observed inputs $\mathbf{X} \in \mathbb{R}^{n \times d_0}$ and outputs $\mathbf{y} \in \mathbb{R}^n$, the generative model is:

$$\begin{aligned}
\mathbf{F}^{(1)} &= f^{(1)}(\mathbf{X}), & f^{(1)} &\sim \mathcal{GP}(0, k^{(1)}), \\
\mathbf{F}^{(2)} &= f^{(2)}(\mathbf{F}^{(1)}), & f^{(2)} &\sim \mathcal{GP}(0, k^{(2)}), \\
&\vdots \\
\mathbf{F}^{(L)} &= f^{(L)}(\mathbf{F}^{(L-1)}), & f^{(L)} &\sim \mathcal{GP}(0, k^{(L)}), \\
\mathbf{y} &= \mathbf{F}^{(L)} + \boldsymbol{\epsilon}, & \boldsymbol{\epsilon} &\sim \mathcal{N}(\mathbf{0}, \sigma_n^2 \mathbf{I}),
\end{aligned}$$

where $\mathbf{F}^{(\ell)} \in \mathbb{R}^{n \times d_\ell}$ contains function evaluations at layer ℓ for all n data points, and d_ℓ is the output dimensionality of layer ℓ.

The covariance matrix at layer ℓ is constructed from kernel evaluations at the *random* outputs of the previous layer:

$$\left[K^{(\ell)}\right]_{ij} = k^{(\ell)}\left(\mathbf{f}_i^{(\ell-1)}, \mathbf{f}_j^{(\ell-1)}\right),$$

where $\mathbf{f}_i^{(\ell-1)} \in \mathbb{R}^{d_{\ell-1}}$ is the i-th row of $\mathbf{F}^{(\ell-1)}$. When conditioned on the outputs from layer $\ell - 1$, the distribution of $\mathbf{F}^{(\ell)}$ remains multivariate Gaussian:

$$p\left(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)}\right) = \mathcal{N}\left(\mathbf{F}^{(\ell)} \mid \mathbf{0}, K^{(\ell)}\right).$$

When $\mathbf{F}^{(\ell-1)}$ is held fixed, this is identical to a standard GP prior and layer ℓ behaves exactly like a regular GP evaluated at those inputs. The difficulty arises only when $\mathbf{F}^{(\ell-1)}$ is random as the covariance matrix $K^{(\ell)}$ then becomes a random variable itself, and integrating over it no longer gives a closed form solution.

The directed graph structure implies that each layer depends only on its immediate predecessor (which is a Markov property), leading to a factorised joint prior:

$$p\left(\mathbf{F}^{(1)}, \ldots, \mathbf{F}^{(L)} \mid \mathbf{X}\right) = p\left(\mathbf{F}^{(1)} \mid \mathbf{X}\right) \prod_{\ell=2}^{L} p\left(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)}\right).$$

This follows from applying the chain rule and noting that $\mathbf{F}^{(\ell)} \perp \mathbf{F}^{(1:\ell-2)}, \mathbf{X} \mid \mathbf{F}^{(\ell-1)}$. Once you know what layer $\ell-1$ output, knowledge of all earlier layers gives no additional information about layer ℓ. Practically, this means one can sample through the model one layer at a time, discarding each layer's outputs once they have been passed forward. The likelihood connects the final layer to observations via $p(\mathbf{y} \mid \mathbf{F}^{(L)}) = \mathcal{N}(\mathbf{y} \mid \mathbf{F}^{(L)}, \sigma_n^2 \mathbf{I})$.

Equivalently, defining the composed function $f = f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)}$:

$$\mathbf{y} = f(\mathbf{X}) + \boldsymbol{\epsilon} = f^{(L)}(f^{(L-1)}(\cdots f^{(1)}(\mathbf{X}))) + \boldsymbol{\epsilon}.$$

When layers have output dimension $d_\ell > 1$, the standard approach uses independent GPs per output dimension:

$$\mathbf{F}^{(\ell)} = \left[\mathbf{f}^{(\ell,1)}, \ldots, \mathbf{f}^{(\ell,d_\ell)}\right], \quad f^{(\ell,j)} \overset{\text{ind}}{\sim} \mathcal{GP}\left(0, k^{(\ell)}\right).$$

This leads to $\sum_{\ell=1}^{L} n \cdot d_\ell = O(nLd)$ total latent variables.

Figure 4 illustrates the compositional structure of a two-layer DGP. The first layer applies an aggressive input warp that stretches the centre of input space while compressing the tails; the second layer applies a stationary GP to that warped space. The composed function becomes non-stationary, varying rapidly in the stretched centre and slowly in the compressed tails. This is behaviour that no single stationary GP could replicate.

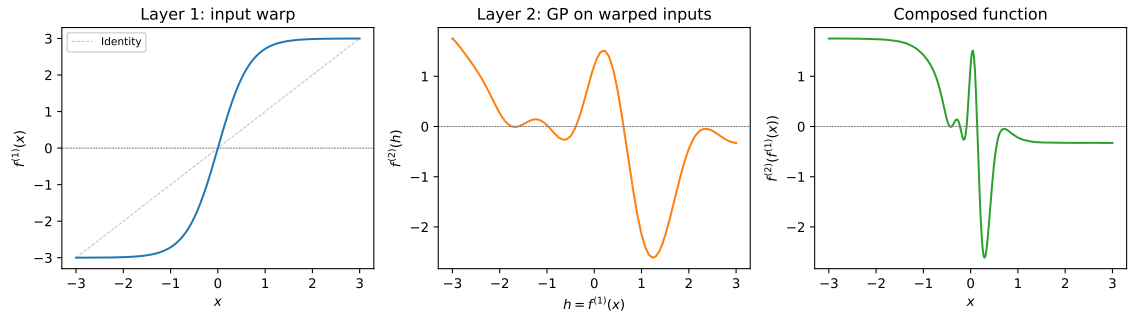


Figure 4: Compositional structure of a two-layer DGP. Left: the first-layer function $f^{(1)}$ is a steep sigmoid warp. Near $x = 0$ the slope is large (inputs map to a wide range of h) while near ± 3 it flattens (inputs cluster at the extremes of h). Centre: the second-layer GP $f^{(2)}$ is stationary and uniform on the warped axis h. Right: the composed function $f^{(2)} \circ f^{(1)}$ is non-stationary: it varies rapidly near $x = 0$ where the warp stretches input space, and is nearly flat near $x = \pm 3$ where the warp compresses it. Neither layer alone could produce this varying-lengthscale behaviour.

The expressiveness of DGPs over standard GPs comes from this composition (Damianou and Lawrence 2013; Dunlop et al. 2018). Even when each layer uses a stationary kernel, the composed function yields a non-stationary process. Consider the covariance of $g = f^{(2)} \circ f^{(1)}$:

$$\mathrm{Cov}[g(\mathbf{x}), g(\mathbf{x}')] = \mathbb{E}_{f^{(1)}}\left[k^{(2)}\left(f^{(1)}(\mathbf{x}), f^{(1)}(\mathbf{x}')\right)\right]. \tag{11}$$

This expectation depends on marginal distributions of $f^{(1)}(\mathbf{x})$ and $f^{(1)}(\mathbf{x}')$, which changes with $\mathbf{x}, \mathbf{x}'$ through $k^{(1)}$. The composed DGP covariance is therefore not just a function of $\mathbf{x} - \mathbf{x}'$, which makes the process non-stationary. This input-dependent structure lets DGPs model functions with varying smoothness and lengthscale across input space. To see why this is useful, imagine fitting a signal that is smooth in some regions and rapidly varying in others: a single GP with a fixed lengthscale must compromise, while a DGP can learn to "stretch" the input space in smooth regions and "compress" it in complex regions through $f^{(1)}$, which gives the second layer an effectively adapted coordinate system rather than the raw inputs to work with. (Dunlop et al. 2018)

4.2 Intractability of Exact Inference

For standard single-layer GPs, the posterior $p(f_* \mid \mathbf{X}, \mathbf{y})$ and marginal likelihood $p(\mathbf{y} \mid \mathbf{X})$ give closed-form expressions due to Gaussian conjugacy (Rasmussen and Williams 2006). In DGPs this is lost because composition of GPs produces a non-Gaussian joint distribution.

The marginal likelihood requires integrating out all latent functions:

$$p(\mathbf{y} \mid \mathbf{X}) = \int p(\mathbf{y} \mid \mathbf{F}^{(L)}) \prod_{\ell=1}^{L} p(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)})\, d\mathbf{F}^{(1)} \cdots d\mathbf{F}^{(L)},$$

where $\mathbf{F}^{(0)} = \mathbf{X}$. This integral has no closed-form solution for $L \geq 2$ (unless the kernel is linear, which reduces the composition to a single-layer GP). The fundamental obstacle is that the covariance matrix $K^{(\ell)}$ at each layer is a nonlinear function of $\mathbf{F}^{(\ell-1)}$. These are the very variables being integrated over, so the integral cannot be reorganised into the product-of-Gaussians form that makes single-layer GP inference tractable.

To see why, consider a two-layer DGP. The covariance matrix $K^{(2)}$ depends on $\mathbf{F}^{(1)}$ through $[K^{(2)}]_{ij} = k^{(2)}(f_i^{(1)}, f_j^{(1)})$. For the squared exponential kernel:

$$k^{(2)}(h_1, h_2) = \sigma_f^2 \exp\left(-\frac{(h_1 - h_2)^2}{2\ell^2}\right),$$

this is a nonlinear function of the Gaussian random variables $(f_i^{(1)}, f_j^{(1)})$. The integral

$$\int \mathcal{N}(\mathbf{F}^{(2)} \mid \mathbf{0}, K^{(2)}(\mathbf{F}^{(1)}))\mathcal{N}(\mathbf{F}^{(1)} \mid \mathbf{0}, K^{(1)})\, d\mathbf{F}^{(1)}$$

is a mixture of Gaussians with continuously varying covariance. Intuitively, for every possible realisation of $\mathbf{F}^{(1)}$ there is a different Gaussian over $\mathbf{F}^{(2)}$; marginalising out $\mathbf{F}^{(1)}$ means averaging together infinitely many Gaussians with different shapes, producing a result that is heavier-tailed and more complex than any individual Gaussian component. These do not have a closed-form expression, and the resulting distribution over $\mathbf{F}^{(2)}$ is non-Gaussian.

As well as the intractability of the marginal likelihood, the resulting joint distribution over the final-layer latent values is non-Gaussian. Consider a two-layer DGP with first-layer outputs $h_1 = f^{(1)}(x_1)$, $h_2 = f^{(1)}(x_2)$. From the GP prior:

$$\begin{bmatrix} h_1 \\ h_2 \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} k_{11}^{(1)} & k_{12}^{(1)} \\ k_{12}^{(1)} & k_{22}^{(1)} \end{bmatrix}\right).$$

Writing $f_i^{(2)} = f^{(2)}(h_i)$ for the second-layer latent at input h_i, conditioning on (h_1, h_2) gives:

$$\begin{bmatrix} f_1^{(2)} \\ f_2^{(2)} \end{bmatrix} \middle| h_1, h_2 \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} k^{(2)}(h_1, h_1) & k^{(2)}(h_1, h_2) \\ k^{(2)}(h_1, h_2) & k^{(2)}(h_2, h_2) \end{bmatrix}\right).$$

The marginal distribution

$$p(f_1^{(2)}, f_2^{(2)}) = \int p(f_1^{(2)}, f_2^{(2)} \mid h_1, h_2)\, p(h_1, h_2)\, dh_1\, dh_2$$

is a continuous mixture of Gaussians with varying covariance matrices. Since $k^{(2)}(h_1, h_2)$ is a nonlinear function of (h_1, h_2), this mixture is not Gaussian, so the output $\mathbf{y} = \mathbf{f}^{(2)} + \boldsymbol{\epsilon}$ is also non-Gaussian.

The posterior over all latent functions is:

$$p(\mathbf{F}^{(1:L)} \mid \mathbf{X}, \mathbf{y}) = \frac{p(\mathbf{y} \mid \mathbf{F}^{(L)}) \prod_{\ell=1}^{L} p(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)})}{p(\mathbf{y} \mid \mathbf{X})}.$$

Although the numerator can be evaluated for any given setting of the latent values, the denominator (the marginal likelihood) requires integrating over every layer and has no closed form, so exact posterior computation is infeasible.

Even numerical integration is impractical. The integration is over $\sum_{\ell=1}^{L} nd_\ell$ continuous variables, which cannot be factored across layers due to kernel dependencies. This motivates the development of approximate inference methods for DGP learning (Damianou and Lawrence 2013; Salimbeni and Deisenroth 2017).

5.0 Learning in GP and DGP Models

The previous section showed that DGPs offer richer modelling capacity than standard GPs, but at the cost of tractable inference. This section develops the framework for learning in both settings.

We briefly revisit hyperparameter learning in standard GPs, focusing on how the marginal likelihood arises by integrating out function values, to identify the structural property that is lost in DGPs. We then discuss variational inference for approximating intractable posteriors, and build to sparse variational methods that reduce computation from $O(n^3)$ to $O(nm^2)$, and finally extend these ideas to DGPs by using doubly stochastic variational inference (Salimbeni and Deisenroth 2017) to train DGPs at scale.

5.1 Learning in Standard GPs

As shown in Section 3.3, the GP hyperparameters $\boldsymbol{\theta}$ are learned by maximising the LML. It is worth seeing explicitly how this LML arises by integrating out the latent function values $\mathbf{f}$.

Starting from the GP prior $\mathbf{f} \mid \mathbf{X} \sim \mathcal{N}(\mathbf{0}, K_{\boldsymbol{\theta}})$ and likelihood $\mathbf{y} \mid \mathbf{f} \sim \mathcal{N}(\mathbf{f}, \sigma_n^2 \mathbf{I})$, Gaussian–Gaussian conjugacy (Bernardo and Smith 2000) allows the integral over function values to be computed in closed form:

$$p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\theta}) = \int p(\mathbf{y} \mid \mathbf{f}) p(\mathbf{f} \mid \mathbf{X}, \boldsymbol{\theta}) \, d\mathbf{f} = \mathcal{N}(\mathbf{y} \mid \mathbf{0}, K_{\boldsymbol{\theta}} + \sigma_n^2 \mathbf{I}),$$

giving the LML of Equation 8. As the product of two Gaussians is proportional to a Gaussian, the $\mathbf{f}$-integral collapses to a single multivariate normal with covariance $K_{\boldsymbol{\theta}} + \sigma_n^2 \mathbf{I}$.

Hyperparameters are then found by gradient ascent using the gradient derived in Section 3.5. Each step requires a Cholesky decomposition of $K_{\boldsymbol{\theta}} + \sigma_n^2 \mathbf{I}$, at $O(n^3)$ cost with $O(n^2)$ storage (Rasmussen and Williams 2006). This is feasible for small datasets but becomes prohibitive as n grows.

5.2 Variational Inference

When exact inference is intractable, variational inference allows for approximations (Jordan et al. 1999; Blei et al. 2017). The idea is to approximate the true posterior $p(\mathbf{z} \mid \mathbf{y})$ with a simpler, tractable distribution $q(\mathbf{z})$, which is chosen to minimise the Kullback–Leibler (KL) divergence between q and p. For latent variables $\mathbf{z}$ and observations $\mathbf{y}$, the log marginal likelihood separates into the evidence lower bound (ELBO) and KL divergence:

$$\log p(\mathbf{y}) = \mathcal{L}(q) + \mathrm{KL}(q(\mathbf{z}) \| p(\mathbf{z} \mid \mathbf{y})), \tag{12}$$

where the evidence lower bound (ELBO) is:

$$\mathcal{L}(q) = \mathbb{E}_{q(\mathbf{z})}[\log p(\mathbf{y}, \mathbf{z})] - \mathbb{E}_{q(\mathbf{z})}[\log q(\mathbf{z})] = \mathbb{E}_{q(\mathbf{z})}[\log p(\mathbf{y} \mid \mathbf{z})] - \mathrm{KL}(q(\mathbf{z}) \| p(\mathbf{z})). \tag{13}$$

Since KL divergence is non-negative, $\mathcal{L}(q) \leq \log p(\mathbf{y})$ with equality when $q(\mathbf{z}) = p(\mathbf{z} \mid \mathbf{y})$. Inference therefore becomes an optimisation problem: since $\log p(\mathbf{y})$ does not depend on q, maximising the ELBO is equivalent to minimising the KL divergence to the true posterior.

To show Equation 12, start from the definition of KL divergence:

$$\begin{aligned} \mathrm{KL}(q \| p) &= \mathbb{E}_{q(\mathbf{z})}\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z} \mid \mathbf{y})}\right] = \mathbb{E}_{q(\mathbf{z})}[\log q(\mathbf{z})] - \mathbb{E}_{q(\mathbf{z})}[\log p(\mathbf{z} \mid \mathbf{y})] \\ &= \mathbb{E}_{q(\mathbf{z})}[\log q(\mathbf{z})] - \mathbb{E}_{q(\mathbf{z})}[\log p(\mathbf{y}, \mathbf{z})] + \log p(\mathbf{y}). \end{aligned}$$

Rearranging this gives Equation 12.

The second form Equation 13 shows the ELBO's structure. The expected log-likelihood $\mathbb{E}_q[\log p(\mathbf{y} \mid \mathbf{z})]$ pushes q to concentrate where the data is explained well, while the KL term $\mathrm{KL}(q \| p(\mathbf{z}))$ pushes q toward the prior, preventing overfitting

(Bishop 2006). This trade-off mirrors the data-fit versus complexity balance we see in exact Bayesian inference.

For the models in this dissertation, variational inference serves two purposes. Firstly, it allows for scaling to large datasets. Secondly, it allows for handling the hierarchical structure of DGPs where function values at each layer cannot be analytically integrated out which is discussed later (Section 5.4). The latent variables $\mathbf{z}$ correspond to function values $\mathbf{f}$, and the variational distribution is usually chosen to be Gaussian:

$$q(\mathbf{f}) = \mathcal{N}(\mathbf{f} \mid \mathbf{m}, S),$$

with variational parameters $\mathbf{m} \in \mathbb{R}^n$ and $S \in \mathbb{R}^{n \times n}$ (positive definite). This balances expressiveness against computational cost. The expectations under q can often be computed analytically for standard likelihood functions.

With this Gaussian choice, the KL term has closed form:

$$\mathrm{KL}(q(\mathbf{f}) \| p(\mathbf{f})) = \frac{1}{2}\left[\mathrm{tr}(K^{-1}S) + \mathbf{m}^\top K^{-1}\mathbf{m} - n + \log|K| - \log|S|\right],$$

where K is the prior covariance.

Each of these terms has an interpretable role: $\mathbf{m}^\top K^{-1}\mathbf{m}$ penalises the variational mean for straying from zero, measured as a Mahalanobis distance under the prior covariance K; $\mathrm{tr}(K^{-1}S)$ penalises the variational covariance for being large relative to K; and $\log|K| - \log|S|$ penalises q for being more concentrated than the prior. The last two terms balance at $S = K$. These together make q pay a KL cost for deviating from the prior, which helps prevent overfitting.

However, optimisation over a full-rank matrix $S \in \mathbb{R}^{n \times n}$ still involves $O(n^2)$ parameters, which remains problematic for large n.

5.3 Sparse Variational GPs

The $O(n^3)$ cost of standard GP inference becomes prohibitive for large datasets. We can reduce this by using $m \ll n$ *inducing points* that summarise the data, reducing complexity to $O(nm^2)$ (Quinonero-Candela and Rasmussen 2005; Titsias 2009).

Consider inducing inputs $\mathbf{Z} = \{\mathbf{z}_j\}_{j=1}^m$ with function values $\mathbf{u} = f(\mathbf{Z}) \in \mathbb{R}^m$. From the prior, the joint distribution of function values $\mathbf{f}$ at training inputs and inducing values $\mathbf{u}$ is:

$$\begin{bmatrix}\mathbf{f}\\ \mathbf{u}\end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix}K_{ff} & K_{fu}\\ K_{uf} & K_{uu}\end{bmatrix}\right),$$

where $K_{ff} = K(\mathbf{X}, \mathbf{X})$, $K_{fu} = K(\mathbf{X}, \mathbf{Z})$, and $K_{uu} = K(\mathbf{Z}, \mathbf{Z})$. K_{fu} is the cross-covariance between training and inducing points, it encodes how closely each training point correlates with each inducing point. If inducing points are placed where the data concentrates, K_{fu} will be large and the m inducing values will carry most of the information about the n function values.

Titsias (2009) showed that we can treat $\mathbf{Z}$ as variational parameters rather than fixed pseudo-inputs. The key idea is to use the conditional $p(\mathbf{f} \mid \mathbf{u})$ from the prior and place a variational distribution on $\mathbf{u}$:

$$q(\mathbf{f}, \mathbf{u}) = p(\mathbf{f} \mid \mathbf{u})q(\mathbf{u}),$$

where $q(\mathbf{u}) = \mathcal{N}(\mathbf{u} \mid \mathbf{m}, S)$ with variational parameters $\mathbf{m} \in \mathbb{R}^m$ and $S \in \mathbb{R}^{m \times m}$. Rather than placing a variational distribution directly on all n function values $\mathbf{f}$, we only learn a distribution over the m inducing values $\mathbf{u}$ and use the GP conditional $p(\mathbf{f} \mid \mathbf{u})$ to fill in the rest. This reduces the variational parameter count from $O(n^2)$ to $O(m^2)$.

$$p(\mathbf{f} \mid \mathbf{u}) = \mathcal{N}(\mathbf{f} \mid K_{fu}K_{uu}^{-1}\mathbf{u},\ K_{ff} - K_{fu}K_{uu}^{-1}K_{uf}).$$

This is the noise-free GP predictive formula from Section 3.6, with the inducing values $\mathbf{u}$ playing the role of observed data and the training inputs playing the role of test points. The conditional mean $K_{fu}K_{uu}^{-1}\mathbf{u}$ is a weighted sum of inducing values. The conditional variance $K_{ff} - K_{fu}K_{uu}^{-1}K_{uf}$ is the prior variance reduced by the information the inducing points provide and so it vanishes at the inducing locations themselves.

Marginalising out $\mathbf{u}$, the variational distribution over $\mathbf{f}$ is:

$$q(\mathbf{f}) = \int p(\mathbf{f} \mid \mathbf{u})q(\mathbf{u})\, d\mathbf{u} = \mathcal{N}(\mathbf{f} \mid K_{fu}K_{uu}^{-1}\mathbf{m},\ K_{ff} - K_{fu}K_{uu}^{-1}K_{uf} + K_{fu}K_{uu}^{-1}SK_{uu}^{-1}K_{uf}).$$

Compared to the conditional $p(\mathbf{f} \mid \mathbf{u} = \mathbf{m})$, the variational posterior $q(\mathbf{f})$ adds the term $K_{fu}K_{uu}^{-1}SK_{uu}^{-1}K_{uf}$ to the variance which restores the uncertainty lost by treating $\mathbf{u}$ as fixed at $\mathbf{m}$, so $q(\mathbf{f})$ correctly reflects our variational uncertainty about $\mathbf{u}$.

The ELBO for this sparse model is:

$$\mathcal{L} = \mathbb{E}_{q(\mathbf{f})}[\log p(\mathbf{y} \mid \mathbf{f})] - \mathrm{KL}(q(\mathbf{u}) \| p(\mathbf{u})). \tag{14}$$

The KL term has closed form:

$$\mathrm{KL}(q(\mathbf{u}) \| p(\mathbf{u})) = \frac{1}{2}\left[\mathrm{tr}(K_{uu}^{-1} S) + \mathbf{m}^\top K_{uu}^{-1} \mathbf{m} - m + \log |K_{uu}| - \log |S|\right].$$

For Gaussian likelihoods $p(\mathbf{y} \mid \mathbf{f}) = \mathcal{N}(\mathbf{y} \mid \mathbf{f}, \sigma_n^2 \mathbf{I})$, the expected log-likelihood can be computed in closed form, which leads to a fully analytical ELBO that can be optimised against $\mathbf{m}$, S, $\mathbf{Z}$ and $\boldsymbol{\theta}$. The optimal variational parameters are:

$$\begin{aligned} S^{-1} &= K_{uu}^{-1} + \sigma_n^{-2} K_{uu}^{-1} K_{uf} K_{fu} K_{uu}^{-1}, \\ \mathbf{m} &= \sigma_n^{-2} S K_{uu}^{-1} K_{uf} \mathbf{y}. \end{aligned}$$

With inducing points, predictions at test inputs $\mathbf{X}_*$ follow:

$$\begin{aligned} \mathbb{E}[f_*] &= K_{*u} K_{uu}^{-1} \mathbf{m}, \\ \mathrm{Var}[f_*] &= K_{**} - K_{*u} K_{uu}^{-1} (K_{uu} - S) K_{uu}^{-1} K_{u*}. \end{aligned}$$

We now are left with $O(m^3)$ for inverting K_{uu}, $O(nm^2)$ for matrix products involving K_{fu}, and $O(m^2)$ storage for variational parameters. Inducing locations $\mathbf{Z}$ can be jointly optimised with hyperparameters $\boldsymbol{\theta}$ and variational parameters $(\mathbf{m}, S)$ by gradient ascent on the ELBO.

5.3.1 Stochastic Optimisation via Minibatches

Even at $O(nm^2)$ per gradient step, iterating over the full dataset each step becomes expensive for large n. For any factorised likelihood, the expected log-likelihood in Equation 14 breaks apart over individual data points into

$$\mathbb{E}_{q(\mathbf{f})}[\log p(\mathbf{y} \mid \mathbf{f})] = \sum_{i=1}^{n} \mathbb{E}_{q(f_i)}[\log p(y_i \mid f_i)],$$

since $p(\mathbf{y} \mid \mathbf{f}) = \prod_i p(y_i \mid f_i)$ and the marginals $q(f_i) = \mathcal{N}(f_i \mid [K_{fu} K_{uu}^{-1} \mathbf{m}]_i, [K_{ff} - K_{fu} K_{uu}^{-1} (K_{uu} - S) K_{uu}^{-1} K_{uf}]_{ii})$ are available in closed form. The KL term does not depend on the data and can be evaluated once per step rather than per data point for efficiency. This per-point structure means the ELBO can be estimated unbiasedly from a random minibatch $\mathcal{B} \subset \{1, \ldots, n\}$ (Hensman et al. 2013):

$$\hat{\mathcal{L}} = \frac{n}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbb{E}_{q(f_i)}[\log p(y_i \mid f_i)] - \mathrm{KL}(q(\mathbf{u}) \| p(\mathbf{u})).$$

$n/|\mathcal{B}|$ re-scales the minibatch sum so that it turns into an unbiased estimate of the full sum over all n points. Therefore, a minibatch of 100 data points still gives a correctly-scaled estimate for a dataset of 100,000. Each step now costs $O(|\mathcal{B}|m^2 + m^3)$, making gradient-based optimisation feasible even for very large datasets.

This sparse variational framework forms the foundation for scalable DGP inference, with each layer maintaining its own set of inducing points.

5.4 Variational Inference for DGPs

As shown in Section 4.0, the layer-ℓ covariance $K^{(\ell)}$ depends non-linearly on the random outputs $\mathbf{F}^{(\ell-1)}$, so the intermediate layers cannot be integrated out analytically. We therefore extend the sparse variational approximation of Section 5.3 to each layer.

For an L-layer DGP, let $\mathbf{Z}^{(\ell)} \in \mathbb{R}^{m_\ell \times d_{\ell-1}}$ denote inducing inputs at layer ℓ, with corresponding inducing outputs $\mathbf{U}^{(\ell)} = f^{(\ell)}(\mathbf{Z}^{(\ell)}) \in \mathbb{R}^{m_\ell \times d_\ell}$. Following from work by Damianou and Lawrence (2013) and Titsias (2009), the variational distribution is able to factorise across layers:

$$q(\{\mathbf{F}^{(\ell)}, \mathbf{U}^{(\ell)}\}_{\ell=1}^{L}) = \prod_{\ell=1}^{L} p(\mathbf{F}^{(\ell)} \mid \mathbf{U}^{(\ell)}, \mathbf{F}^{(\ell-1)}) q(\mathbf{U}^{(\ell)}), \tag{15}$$

where $q(\mathbf{U}^{(\ell)}) = \mathcal{N}(\mathbf{U}^{(\ell)} \mid \mathbf{M}^{(\ell)}, \mathbf{S}^{(\ell)})$ with variational parameters $\mathbf{M}^{(\ell)} \in \mathbb{R}^{m_\ell \times d_\ell}$ and covariances $\mathbf{S}^{(\ell)}$. Rather than placing a variational distribution over each layer's full $n \times d_\ell$ output matrix, we only learn distributions over the much smaller $m_\ell \times d_\ell$ inducing outputs at each layer. The per-layer GP conditional $p(\mathbf{F}^{(\ell)} \mid \mathbf{U}^{(\ell)}, \mathbf{F}^{(\ell-1)})$ then moves information from inducing values to all training points.

The conditional $p(\mathbf{F}^{(\ell)} \mid \mathbf{U}^{(\ell)}, \mathbf{F}^{(\ell-1)})$ uses the GP prior structure. For layer ℓ with inputs from the previous layer $\mathbf{F}^{(\ell-1)}$ and inducing inputs $\mathbf{Z}^{(\ell)}$:

$$p(\mathbf{F}^{(\ell)} \mid \mathbf{U}^{(\ell)}, \mathbf{F}^{(\ell-1)}) = \mathcal{N}\left(\mathbf{F}^{(\ell)} \mid A^{(\ell)}\mathbf{U}^{(\ell)},\, K_{ff}^{(\ell)} - A^{(\ell)} K_{uu}^{(\ell)} A^{(\ell)\top}\right),$$

where $A^{(\ell)} = K_{fu}^{(\ell)}(K_{uu}^{(\ell)})^{-1}$. The covariance matrices $K_{ff}^{(\ell)}$, $K_{fu}^{(\ell)}$, and $K_{uu}^{(\ell)}$ depend on the random inputs $\mathbf{F}^{(\ell-1)}$, creating dependencies across layers.

The ELBO for DGPs is:

$$\mathcal{L} = \mathbb{E}_{q(\mathbf{F}^{(1:L)})}[\log p(\mathbf{y} \mid \mathbf{F}^{(L)})] - \sum_{\ell=1}^{L} \mathrm{KL}(q(\mathbf{U}^{(\ell)}) \| p(\mathbf{U}^{(\ell)})). \tag{16}$$

There is one data-fit term (the expected log-likelihood at the final layer) and one KL regularisation term per layer, which penalises each layer's variational posterior for deviating from its GP prior, just as there is in the standard GP case. The KL terms have closed form since $q(\mathbf{U}^{(\ell)})$ and $p(\mathbf{U}^{(\ell)})$ are both Gaussian. However, the expected log-likelihood requires integrating over all the layers outputs $\mathbf{F}^{(1:L)}$ which has non-Gaussian dependencies across layers and so cannot be easily evaluated.

The expectation in Equation 16 involves:

$$\mathbb{E}_q[\log p(\mathbf{y} \mid \mathbf{F}^{(L)})] = \int \cdots \int \log p(\mathbf{y} \mid \mathbf{F}^{(L)}) \prod_{\ell=1}^{L} q(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)})\, d\mathbf{F}^{(1)} \cdots d\mathbf{F}^{(L)},$$

where $q(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)})$ is obtained by integrating out $\mathbf{U}^{(\ell)}$:

$$q(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)}) = \int p(\mathbf{F}^{(\ell)} \mid \mathbf{U}^{(\ell)}, \mathbf{F}^{(\ell-1)}) q(\mathbf{U}^{(\ell)})\, d\mathbf{U}^{(\ell)}.$$

This integral is Gaussian:

$$q(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)}) = \mathcal{N}\left(\mathbf{F}^{(\ell)} \mid A^{(\ell)}\mathbf{M}^{(\ell)},\, K_{ff}^{(\ell)} - A^{(\ell)}(K_{uu}^{(\ell)} - \mathbf{S}^{(\ell)})A^{(\ell)\top}\right).$$

Compared to the GP prior $p(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)}) = \mathcal{N}(\mathbf{0}, K_{ff}^{(\ell)})$, this has a non-zero mean (coming from $\mathbf{M}^{(\ell)}$) and lower variance (the term $A^{(\ell)}(K_{uu}^{(\ell)} - \mathbf{S}^{(\ell)})A^{(\ell)\top}$ takes off the uncertainty that is explained by the inducing points, then adds back only what $\mathbf{S}^{(\ell)}$ says is left). While Gaussian in $\mathbf{F}^{(\ell)}$ when $\mathbf{F}^{(\ell-1)}$ is fixed, the overall distribution is non-Gaussian because the covariance matrices $K^{(\ell)}$ depend non-linearly on the random $\mathbf{F}^{(\ell-1)}$.

Damianou and Lawrence (2013) addressed this by further approximating the variational distribution. They introduce variational distributions over the intermediate layer outputs:

$$q(\mathbf{F}^{(\ell)}) = \mathcal{N}(\mathbf{F}^{(\ell)} \mid \boldsymbol{\mu}^{(\ell)}, \boldsymbol{\Sigma}^{(\ell)}),$$

treating $\{\boldsymbol{\mu}^{(\ell)}, \boldsymbol{\Sigma}^{(\ell)}\}$ as additional variational parameters. This "mean-field" approximation breaks the dependencies between layers but introduces $O(nLd)$ additional

parameters and loses the structure that $\mathbf{F}^{(\ell)}$ should be generated from $\mathbf{F}^{(\ell-1)}$ via a GP.

The resulting ELBO becomes:

$$\mathcal{L}_{\text{MF}} = \mathbb{E}_{q(\mathbf{F}^{(L)})}[\log p(\mathbf{y} \mid \mathbf{F}^{(L)})] - \sum_{\ell=1}^{L} \mathbb{E}_{q(\mathbf{F}^{(\ell-1)})} \left[\text{KL}(q(\mathbf{F}^{(\ell)}) \| p(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)}))\right].$$

This approach enabled the original DGP work but suffers from scalability issues (variational parameters grow with n) and may produce poor approximations due to the mean-field assumption of independence between each layer's outputs, which ignores the hierarchical structure of the model.

5.5 Doubly Stochastic Variational Inference

Salimbeni and Deisenroth (2017) introduced an approach to DGP inference that avoids the mean-field approximation over layer outputs, and its limitations. By using Monte Carlo estimation of the nested expectations, it is possible to maintain the exact conditional structure through the GP prior.

The variational distribution keeps the form from Equation 15 but instead of introducing additional variational parameters for $\mathbf{F}^{(\ell)}$, the method passes samples through each of the layers. $\mathbf{F}^{(\ell)}$ is generated from $\mathbf{F}^{(\ell-1)}$ via the GP conditional, and uncertainty is passed through the layers by Monte Carlo sampling from the per-layer marginals $q(\mathbf{F}^{(\ell)} \mid \mathbf{F}^{(\ell-1)})$.

The ELBO decomposes over data points:

$$\mathcal{L} = \sum_{i=1}^{n} \mathbb{E}_{q(f_i^{(L)})}[\log p(y_i \mid f_i^{(L)})] - \sum_{\ell=1}^{L} \text{KL}(q(\mathbf{U}^{(\ell)}) \| p(\mathbf{U}^{(\ell)})). \tag{17}$$

The factorisation follows from $p(\mathbf{y} \mid \mathbf{F}^{(L)}) = \prod_i p(y_i \mid f_i^{(L)})$. The KL terms have closed form since both $q(\mathbf{U}^{(\ell)})$ and $p(\mathbf{U}^{(\ell)})$ are Gaussian (the standard Gaussian KL). However, $\mathbb{E}_{q(f_i^{(L)})}[\log p(y_i \mid f_i^{(L)})]$, requires marginalising over all intermediate layers and so cannot be evaluated in closed form. The distribution $q(f_i^{(L)})$ is obtained by integrating out all the layers:

$$q(f_i^{(L)}) = \int \cdots \int p(f_i^{(L)} \mid \mathbf{U}^{(L)}, f_i^{(L-1)}) q(\mathbf{U}^{(L)}) \cdots p(f_i^{(1)} \mid \mathbf{U}^{(1)}, \mathbf{x}_i) q(\mathbf{U}^{(1)}) \, d\mathbf{U}^{(1)} \cdots d\mathbf{U}^{(L)}.$$

This integral is analytically intractable due to the nonlinear dependencies across layers. The doubly stochastic approach estimates this expectation via Monte Carlo. For a data point $\mathbf{x}_i$, we construct a sample path through all layers. Starting at layer 1, the marginal distribution over $f_i^{(1)}$ given $\mathbf{x}_i$ is:

$$q(f_i^{(1)} \mid \mathbf{x}_i) = \int p(f_i^{(1)} \mid \mathbf{U}^{(1)}, \mathbf{x}_i) q(\mathbf{U}^{(1)})\, d\mathbf{U}^{(1)}.$$

Since $p(f_i^{(1)} \mid \mathbf{U}^{(1)}, \mathbf{x}_i) = \mathcal{N}(f_i^{(1)} \mid k_i^{(1)\top}(K_{uu}^{(1)})^{-1}\mathbf{U}^{(1)},\, k_{ii}^{(1)} - k_i^{(1)\top}(K_{uu}^{(1)})^{-1}k_i^{(1)})$ and $q(\mathbf{U}^{(1)}) = \mathcal{N}(\mathbf{M}^{(1)}, \mathbf{S}^{(1)})$, integrating out $\mathbf{U}^{(1)}$ yields:

$$\begin{aligned}
\mu_i^{(1)} &= k_i^{(1)\top}(K_{uu}^{(1)})^{-1}\mathbf{M}^{(1)}, \\
\sigma_i^{(1)2} &= k_{ii}^{(1)} - k_i^{(1)\top}(K_{uu}^{(1)})^{-1}(K_{uu}^{(1)} - \mathbf{S}^{(1)})(K_{uu}^{(1)})^{-1}k_i^{(1)},
\end{aligned}$$

where $k_i^{(1)} = k^{(1)}(\mathbf{x}_i, \mathbf{Z}^{(1)}) \in \mathbb{R}^{m_1}$ and $k_{ii}^{(1)} = k^{(1)}(\mathbf{x}_i, \mathbf{x}_i)$. The mean $\mu_i^{(1)}$ is the posterior GP mean at $\mathbf{x}_i$, which is a weighted sum of inducing values $\mathbf{M}^{(1)}$, weighted by how correlated $\mathbf{x}_i$ is with each inducing point. The variance $\sigma_i^{(1)2}$ is the prior variance $k_{ii}^{(1)}$ minus the information contributed by the inducing points, plus additional uncertainty from $\mathbf{S}^{(1)}$ which reflects our uncertainty about the inducing outputs.

Sampling $f_i^{(1)} \sim \mathcal{N}(\mu_i^{(1)}, \sigma_i^{(1)2})$ gives a realisation of the first layer's output. For subsequent layers $\ell = 2, \ldots, L$, we condition instead on the sampled value from the layer before. The distribution at layer ℓ given sampled $f_i^{(\ell-1)}$ is:

$$q(f_i^{(\ell)} \mid f_i^{(\ell-1)}) = \int p(f_i^{(\ell)} \mid \mathbf{U}^{(\ell)}, f_i^{(\ell-1)}) q(\mathbf{U}^{(\ell)})\, d\mathbf{U}^{(\ell)},$$

with mean and variance:

$$\begin{aligned}
\mu_i^{(\ell)} &= k^{(\ell)}(f_i^{(\ell-1)}, \mathbf{Z}^{(\ell)})^\top (K_{uu}^{(\ell)})^{-1}\mathbf{M}^{(\ell)}, \\
\sigma_i^{(\ell)2} &= k^{(\ell)}(f_i^{(\ell-1)}, f_i^{(\ell-1)}) \\
&\quad - k^{(\ell)}(f_i^{(\ell-1)}, \mathbf{Z}^{(\ell)})^\top (K_{uu}^{(\ell)})^{-1}(K_{uu}^{(\ell)} - \mathbf{S}^{(\ell)})(K_{uu}^{(\ell)})^{-1}k^{(\ell)}(\mathbf{Z}^{(\ell)}, f_i^{(\ell-1)}).
\end{aligned}$$

The same formula applies as at layer 1, except now that the input is instead the sampled value $f_i^{(\ell-1)}$ from the previous layer rather than the fixed training input $\mathbf{x}_i$. This allows uncertainty to propagate as each layer outputs a Gaussian whose mean and variance depend on a random draw from the layer below, so uncertainty compounds as samples pass through each layer of the DGP.

After sampling through all L layers, we obtain $f_i^{(L)}$ and evaluate $\log p(y_i \mid f_i^{(L)})$. With S independent sample paths, the Monte Carlo estimate of the ELBO is:

$$\hat{\mathcal{L}} = \frac{n}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \frac{1}{S} \sum_{s=1}^{S} \log p(y_i \mid f_i^{(L,s)}) - \sum_{\ell=1}^{L} \mathrm{KL}(q(\mathbf{U}^{(\ell)}) \| p(\mathbf{U}^{(\ell)})),$$

where $\mathcal{B} \subset \{1, \ldots, n\}$ is a minibatch and $f_i^{(L,s)}$ is the s-th sample at the final layer for point i. The factor $n/|\mathcal{B}|$ corrects for minibatch subsampling.

The term "doubly stochastic" refers to two different sources of randomness. We have both minibatch subsampling (stochasticity over data) and Monte Carlo sampling through layers (stochasticity over latent functions). Both maintain unbiasedness of gradient estimates, which is needed for stochastic optimisation to converge. In practice the two sources of noise act independently as increasing the minibatch size $|\mathcal{B}|$ reduces noise from data sampling, and increasing the sample count S reduces noise from layer propagations.

Gradient computation requires differentiating through the sampling operations. A reparametrisation trick (Kingma and Welling 2014; Rezende et al. 2014) makes this tractable as instead of sampling $f \sim \mathcal{N}(\mu, \sigma^2)$ directly, we write $f = \mu + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, 1)$, so that gradients with respect to μ and σ flow through the deterministic transformation while the stochastic node ϵ carries no parameters. By doing this, the randomness is "pushed outside" the computation, so we can treat μ and σ as ordinary differentiable variables.

Computational cost per iteration is $O(|\mathcal{B}|m^2LS)$ for forward propagation of S samples through L layers, plus $O(m^3L)$ for the KL terms. This is fully independent of n, enabling scalability to large datasets. Empirically, a single sample ($S = 1$) often suffices when $|\mathcal{B}|$ is large, as averaging over many data points provides natural variance reduction.

6.0 Model Evaluation

Having developed the theory of DGPs and derived the variational inference framework needed for scalable training, this section evaluates DGP models empirically. We compare GPs and DGPs against standard machine learning baselines across two datasets.

The first experiment uses the Kin8nm robot kinematics dataset (Section 6.2), a regression benchmark with a known non-stationary, compositional structure, where we expect DGPs to outperform single-layer GPs. This establishes whether the additional depth of DGPs translates into measurable gains when the data supports it.

The second experiment applies the same model families to financial credit-risk classification, specifically probability of default on the UCI credit card dataset (Section 6.3). This tests if DGPs add meaningful value on real-world financial data, while also evaluating probability calibration of GP models, which is directly relevant under Basel IRB regulations (*Basel III* 2011).

Across both experiments, the model set contains three classical baselines (a linear model, random forest, and XGBoost) and three GP-family models (Sparse Variational GP (SVGP), DSVGP-2L, DSVGP-3L). Each experiment evaluates performance at multiple training-set sizes to characterise scaling behaviour.

6.1 Evaluation Metrics

Regression performance is measured by RMSE, R^2, and negative log predictive density (NLPD). NLPD evaluates the full predictive distribution: for test observations $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n_*}$ it is

$$\text{NLPD} = -\frac{1}{n_*} \sum_{i=1}^{n_*} \log p(y_i \mid \mathbf{x}_i, \mathcal{D}),$$

where y_i is the observed (noisy) output. For GP models with a Gaussian predictive distribution $\mathcal{N}(y_i \mid \mu_i, \sigma_i^2)$, this reduces to $\frac{1}{n_*} \sum_i \left[\frac{1}{2} \log(2\pi\sigma_i^2) + \frac{(y_i - \mu_i)^2}{2\sigma_i^2}\right]$, where σ_i^2 includes observation noise σ_n^2. Lower NLPD is better. Unlike RMSE, NLPD penalises both inaccurate means and poorly calibrated predictive variance, which makes it one of the primary metrics for assessing uncertainty quality. Classical models (Ridge, Random Forest, XGBoost) do not produce predictive distributions, so NLPD is only reported for GP-family models.

Classification performance is primarily evaluated using ROC-AUC, expected calibration error (ECE), Brier score, and log loss. ECE measures calibration by binning predictions and comparing average predicted probability to observed frequency within each bin; lower ECE indicates better-calibrated probability estimates. F1 at the validation-chosen threshold t^* is also reported as a supplementary metric for classification, showing practical performance at each model's optimal operating point. This threshold selection is described where used in Section 6.3.

Hyperparameters for all models were fixed based on established defaults from prior work rather than tuned via search. For classical baselines, logistic/ridge regression uses default regularisation ($C = 1$), random forest uses 500 trees with balanced class weighting, and XGBoost uses learning rate 0.05 with early stopping on the validation set to select the number of boosting rounds. For GP-family models, SVGP uses 512 inducing points trained for 150 epochs, and DSVGP models use 128 inducing points

per layer with hidden dimension 5. These were chosen as practical defaults for the dataset scales based on available compute resources and converging in a reasonable time frame. All GP/DGP kernel hyperparameters (lengthscales, output variance, noise variance) are learnt jointly with the variational parameters by maximising the ELBO during training.

6.2 Regression Benchmark: Kin8nm

For the first experiment, we evaluate the same GP and DGP architectures on the Kin8nm dataset, a standard regression benchmark from the UCI repository that has been widely used in the DGP literature (Salimbeni and Deisenroth 2017; Damianou and Lawrence 2013).

Kin8nm contains 8,192 observations of the forward kinematics of an 8-link robot arm. There are eight input features which are joint angles, and the task is to predict the distance of the end-effector from a target point. The mapping from joint angles to this target distance is a composition of many trigonometric transformations, which should be the kind of hierarchical non-linearity where the intermediate representations we have in the DGP layers are able to improve performance over standard GPs.

6.2.1 Experimental Design

The 8,192 observations were split into a fixed test set of 1,639 observations (20%) with the remaining 6,553 observations forming the development pool. Three training tiers were evaluated:

- Small ($N_{\text{dev}} = 500$): 425 training, 75 validation.
- Medium ($N_{\text{dev}} = 3{,}000$): 2,550 training, 450 validation.
- Large ($N_{\text{dev}} = 6{,}553$): 5,570 training, 983 validation (full development set).

All features were standardised to zero mean and unit variance, fit on training data only. We train: Ridge Regression, Random Forest (500 trees), XGBoost (with early stopping), SVGP (512 inducing points, 150 epochs), DSVGP-2L (128 inducing points per layer, hidden dimension 5), and DSVGP-3L. GP and DGP models use a Gaussian likelihood for regression. Classical models do not provide predictive distributions, so NLPD is reported only for the GP/DGP models.

6.2.2 Results at Full Scale

Table 1: Regression results on Kin8nm at full scale ($N_{\text{train}} = 5{,}570$, $N_{\text{test}} = 1{,}639$). Bold entries indicate best values. Lower RMSE, MAE, and NLPD are better; higher R^2 is better.

Model	RMSE	MAE	R^2	NLPD	Time (s)
Ridge Regression	0.2023	0.1626	0.408	-	0.001
Random Forest	0.1482	0.1187	0.683	-	0.72
XGBoost	0.1190	0.0902	0.795	-	1.68
SVGP	0.0783	0.0601	0.911	−1.116	36.6
DSVGP-2L	**0.0699**	**0.0541**	**0.929**	**−1.241**	36.2
DSVGP-3L	0.0759	0.0583	0.917	−1.158	152.4

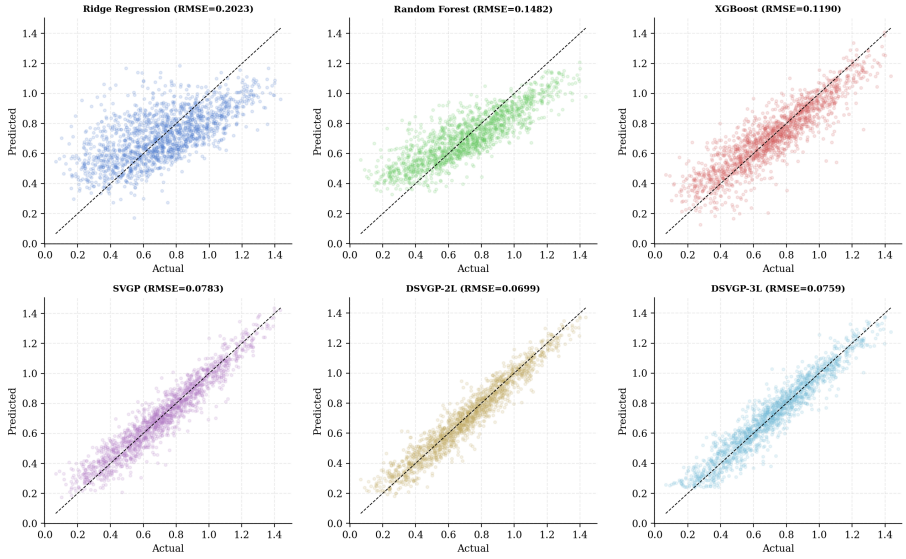


Figure 5: Predicted vs actual values at full scale. DSVGP-2L shows the tightest clustering around the diagonal, with RMSE = 0.0699 vs SVGP's 0.0783.

The Kin8nm results show a clear advantage for the deeper architecture. DSVGP-2L achieves RMSE = 0.0699 versus SVGP's 0.0783, a 10.7% reduction, with $R^2 = 0.929$ versus 0.911. The NLPD improvement (−1.241 vs −1.116, with the more negative value showing better performance) shows that the DGP not only predicts more accurately but also provides better-calibrated predictive uncertainty. In this dataset, additional GP depth does translate into a consistent performance increase.

Kin8nm's target is a composition of trigonometric functions of the joint angles, creating a non-stationary surface where the function's behaviour changes across

the input space. The DGP's hidden layer learns an intermediate representation that effectively warps the input space, making the final-layer mapping smoother (Damianou and Lawrence 2013). A single RBF kernel with a fixed lengthscale cannot adapt to the varying smoothness across different joint-angles, which likely limits the single-layer SVGP's accuracy.

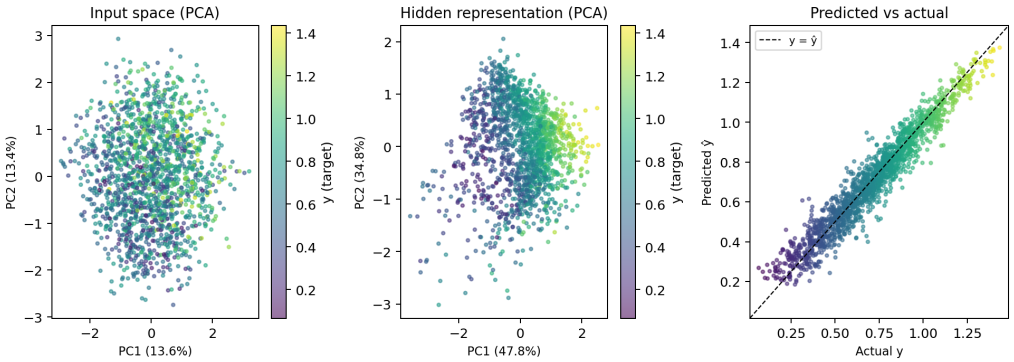


Figure 6: Input space (left) vs DSVGP-2L hidden representation (centre), both PCA-projected to 2D and coloured by target y. The hidden layer reorganises the test-set points so that the target gradient becomes nearly monotone along the first principal component, which is evidence of the learned input-space warping. The right panel shows the resulting predictions.

Figure 6 makes this mechanism clear. In the left panel, the 8-dimensional joint-angle inputs are projected to 2D by PCA: the colour encoding of y is scattered with no clear directional gradient, with low and high output values interleaved throughout the space. The first two principal components of the raw input capture only 27.2% of the variance between them, and neither aligns with the target. In some regions a small joint-angle change produces a large end-effector displacement and in others it barely moves the output, so "similar inputs have similar outputs" holds only locally in the original input space.

The centre panel shows the same test points after passing through DSVGP-2L's first GP layer. In this 5-dimensional hidden representation, projected to 2D by PCA, the colour gradient is now almost entirely ordered along the first principal component, with dark purple (low y) at one end, yellow (high y) at the other. The first PC of the hidden space alone captures 47.8% of the variance, more than three times the input-space PC1's 13.6%, and it is tightly correlated with the target. The hidden layer has learnt a smooth, non-linear warping of the input space in which observations with similar outputs lie close together.

With this warped space in place, the final GP layer only has to fit a smooth, stationary function, a task well suited to a standard RBF kernel. The SVGP instead must approximate a globally non-stationary function in the original joint-angle space with

a single, globally uniform lengthscale, which limits its accuracy regardless of how many inducing points or training epochs are used.

DSVGP-3L performs worse than DSVGP-2L on every metric (RMSE 0.0759, R^2 0.917 vs 0.929, and worse NLPD). As Figure 6 shows, a single hidden layer already makes the target almost monotone along one dimension, so a third layer has little additional structure to learn while adding optimisation difficulty in a larger parameter space.

6.2.3 Scaling Behaviour

Table 2: RMSE and R^2 scaling across training tiers. S = small (425 train), M = medium (2,550 train), L = large (5,570 train).

Model	RMSE (S)	RMSE (M)	RMSE (L)	R^2 (S)	R^2 (M)	R^2 (L)
Ridge Regression	0.2040	0.2024	0.2023	0.398	0.408	0.408
Random Forest	0.1892	0.1629	0.1482	0.483	0.617	0.683
XGBoost	0.1826	0.1389	0.1190	0.518	0.721	0.795
SVGP	**0.1742**	**0.0844**	0.0783	**0.562**	**0.897**	0.911
DSVGP-2L	0.2011	0.0885	**0.0699**	0.416	0.887	**0.929**
DSVGP-3L	0.2628	0.0887	0.0759	0.002	0.886	0.917

With small training sets, SVGP leads (RMSE = 0.1742) while DSVGP-2L (0.2011) and DSVGP-3L (0.2628) lag behind. The deeper architectures are not able to learn useful intermediate representations from just 425 observations. DSVGP-3L's $R^2 = 0.002$ at small scale demonstrates a pathological and near-constant prediction failure. This is consistent with known pathologies of deep GP priors, whose representations degenerate as depth increases so that functions vary along only a few input directions (Duvenaud et al. 2014); with limited data, this makes deeper models harder to fit. A contributing architectural factor is the use of a constant mean function for hidden layers rather than a linear identity mean, which is an approach shown to mitigate small-data instability in deeper configurations (Salimbeni and Deisenroth 2017). With limited data, the optimisation landscape becomes difficult to navigate, and the model can get stuck in poor local minima where the hidden layers fail to learn meaningful representations.

At medium scale, the gap closes: SVGP = 0.0844, DSVGP-2L = 0.0885, DSVGP-3L = 0.0887. At full scale, DSVGP-2L overtakes SVGP (Table 1). This shows that DGPs underperform with limited data but outperform when there's sufficient scale. The key finding is that depth helps when there is both sufficient data and a non-stationary, compositional structure in the target function.

6.3 Credit Card Probability of Default

GP-based methods have been applied to credit-risk and credit-rating problems (Mahajan et al. 2022; Huang 2011), volatility forecasting (Munoz-Gonzalez et al. 2014), and market-risk approximation (Lehdili et al. 2025), but direct DGP applications remain uncommon. Basel III makes banks' capital requirements explicitly risk-sensitive, so reliable probability of default (PD) estimates are critical for regulatory compliance (*Basel III* 2011). Under the IRB Advanced Approach, PD models must demonstrate high discriminative power (correctly ranking borrowers by risk) and accurate probability calibration (predicted default rates must track observed frequencies across rating grades) (European Banking Authority 2017; International Accounting Standards Board 2014). We therefore evaluate whether GP and DGP classifiers meet these regulatory criteria on a realistic credit-risk benchmark, and whether DGP depth adds discriminative value over single-layer GPs.

The benchmark is the UCI default of credit card clients dataset (Yeh and Lien 2009), containing 30,000 observations and 23 predictor variables: credit limit, demographics, six months of repayment-status history, and corresponding billing and repayment amounts from April–September 2005. The target indicates default on the October 2005 payment.

6.3.1 Data Preparation and Experimental Design

Undocumented education and marriage codes were collapsed into an "other" category, skewed monetary variables (bill amounts and repayment amounts) were log-transformed to reduce leverage from extreme values, categorical variables were one-hot encoded with `drop_first=True`, and all continuous predictors were standardised to zero mean and unit variance, fit on the training split only to prevent leakage (see Appendix 1.5.2 for the implementation). The full 30,000-observation dataset was first split into a fixed held-out test set of 6,000 observations (20%), stratified by the target variable to ensure equal default rates across all splits. The remaining 24,000 observations form the development pool. This ensures all models across sizes are evaluated on the same test set, giving a fair comparison across training-set sizes.

- Small: ($N_{\text{dev}} = 500$): 425 training, 75 validation observations.
- Medium: ($N_{\text{dev}} = 5{,}000$): 4,250 training, 750 validation observations.
- Large: ($N_{\text{dev}} = 24{,}000$): 20,400 training, 3,600 validation observations (full development set).

The validation split (15% of dev) serves two purposes: XGBoost early stopping and F1-optimal threshold selection for all models. Each model is first trained on the training set only and evaluated on the validation set to determine its optimal classification threshold t^*. All models are then retrained on the combined train+validation set and evaluated on the held-out test set using the validation-chosen threshold. The test set default rate is 22.1% across all tiers.

6.3.2 Model Set and Training Configuration

The model set and training protocol are consistent with Section 6.2, adapted for binary classification with a Bernoulli likelihood. Three classical baselines are used: logistic regression (replacing Ridge regression from Kin8nm for the classification setting), random forest, and XGBoost. The same three GP-family models are used: SVGP, DSVGP-2L, and DSVGP-3L. All models are trained at each of the three training set sizes (small, medium, large) to characterise scaling behaviour.

6.3.3 Results at Full Scale (Large Tier)

Table 3: Model comparison at full scale ($N_{\text{train}} = 20{,}400$, $N_{\text{test}} = 6{,}000$). Threshold t^* selected on the validation set to maximise F1; all threshold-dependent metrics evaluated on the held-out test set at this fixed threshold. Bold entries indicate leading values.

Model	ROC-AUC	F1 (t^*)	t^*	ECE	Brier	Log Loss	Time (s)
Logistic Regression	0.739	0.500	0.50	0.225	0.199	0.590	0.03
Random Forest	0.769	0.532	0.35	0.087	0.146	0.460	4.85
XGBoost	**0.779**	0.536	0.50	0.196	0.177	0.539	0.24
SVGP	0.778	**0.546**	0.30	**0.010**	**0.135**	**0.430**	160.4
DSVGP-2L	0.777	0.540	0.30	0.014	**0.135**	**0.430**	197.0

Model	ROC-AUC	F1 (t^*)	t^*	ECE	Brier	Log Loss	Time (s)
DSVGP-3L	0.776	0.544	0.30	0.021	0.136	0.433	706.5

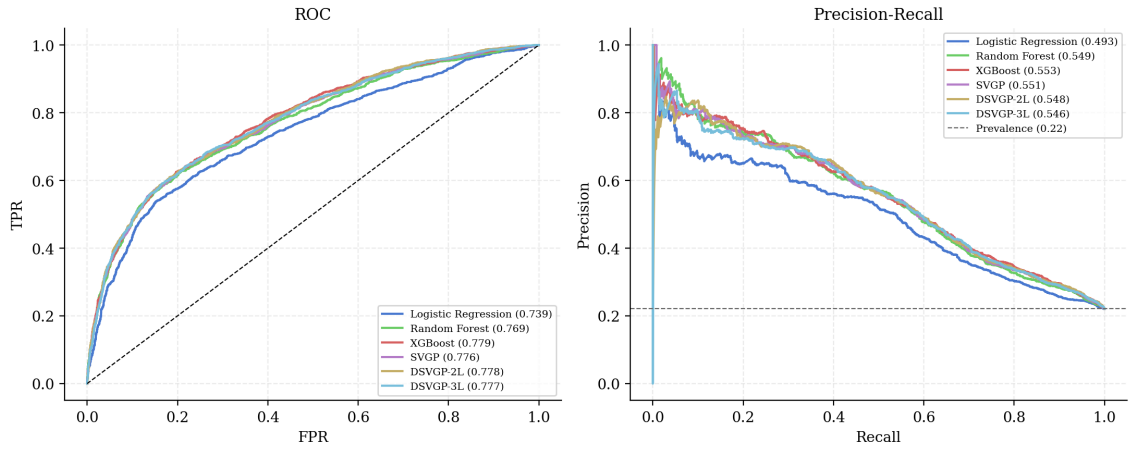


Figure 7: ROC and precision-recall curves at full scale. XGBoost narrowly leads on ROC-AUC (0.779); GP-family models form the second group at 0.776–0.778.

XGBoost leads marginally on ROC-AUC (0.779), with the GP family following at 0.776–0.778, a gap of 0.003 that is small relative to the width of each model's bootstrap CI (±0.015, Appendix 1.8.2) and so well within the test-set's expected variability. At validation-chosen thresholds, performance converges with SVGP = 0.546 ($t^* = 0.30$), and XGBoost = 0.536 ($t^* = 0.50$). The GP models' lower optimal thresholds reflect the calibration. Given the 22.1% base rate of default, properly calibrated probabilities will naturally be clustered below 0.5, which shifts the optimal F1 threshold downward compared to less calibrated models.

The standout result is probability calibration: GP models achieve ECE = 0.010–0.021 versus 0.087–0.225 for classical baselines, a 4–22× improvement over uncalibrated classical outputs. Tree-based models compress their score distributions through regularisation, leading to raw probability estimates that are systematically pulled away from the extremes (Niculescu-Mizil and Caruana 2005). Post-hoc recalibration (Platt scaling (Platt 1999) or isotonic regression) is standard practice to correct this before deployment; GP models achieve this alignment natively, without that step. Under Basel IRB, PD estimates must track observed default rates across rating grades (European Banking Authority 2017), and point-in-time test-set calibration, as measured here, is only one small part of that requirement. These results therefore

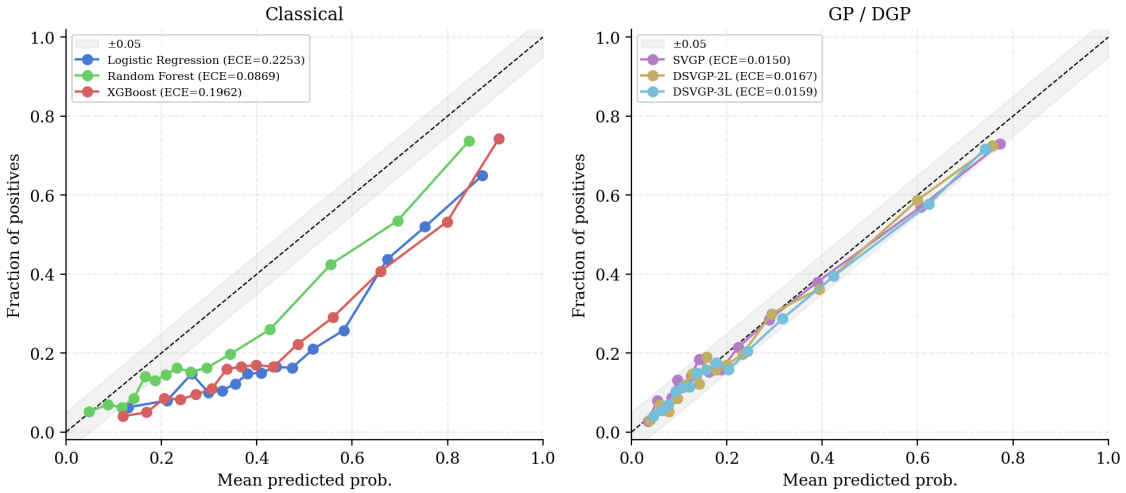


Figure 8: Reliability diagrams at full scale. GP-family models track the diagonal closely (ECE ≤ 0.021); logistic regression and XGBoost show substantial miscalibration (ECE > 0.19).

do not show that GPs meet full regulatory requirements, only that they remove one standard recalibration step.

6.3.4 Scaling Behaviour Across Training Set Sizes

Table 4: Scaling of ROC-AUC and ECE across training tiers. S = small (425 train), M = medium (4,250 train), L = large (20,400 train). All evaluated on the same 6,000-observation test set.

Model	ROC-AUC (S)	ROC-AUC (M)	ROC-AUC (L)	ECE (S)	ECE (M)	ECE (L)
Logistic Regression	0.716	0.737	0.739	0.205	0.229	0.225
Random Forest	**0.746**	0.760	0.769	0.091	0.091	0.087
XGBoost	0.734	0.758	**0.779**	0.164	0.188	0.196
SVGP	0.743	0.766	0.778	0.027	0.013	**0.010**
DSVGP-2L	0.744	**0.768**	0.777	0.023	0.016	0.014
DSVGP-3L	0.523	0.767	0.776	**0.005**	**0.012**	0.021

Table 5: F1 at validation-chosen threshold t^* (shown in parentheses) across training tiers. GP models consistently select lower thresholds.

Model	F1 (S), t^*	F1 (M), t^*	F1 (L), t^*
Logistic Regression	0.475 (0.45)	0.497 (0.55)	0.500 (0.50)
Random Forest	**0.503** (0.40)	**0.530** (0.45)	0.532 (0.35)
XGBoost	0.477 (0.40)	0.519 (0.55)	0.536 (0.50)
SVGP	0.501 (0.30)	0.522 (0.35)	**0.546** (0.30)
DSVGP-2L	0.498 (0.30)	**0.530** (0.30)	0.540 (0.30)
DSVGP-3L	0.362 (0.05)	0.517 (0.35)	0.544 (0.30)

ROC and precision-recall curves and reliability diagrams at each training tier are provided in Appendix 1.6.

All models improve with more data, but the patterns differ by family. XGBoost shows the strongest discriminative scaling improvements (+0.045 ROC-AUC from small to large), while GP models converge toward XGBoost at large scale. DSVGP-3L collapses at small scale (ROC-AUC = 0.523, F1 = 0.362), a clear instance of the same small-data pathology observed on the Kin8nm dataset. Note that the anomalously low ECE = 0.005 for DSVGP-3L at small scale does not indicate good model performance as a model that consistently predicts near the base rate (~22.1% default probability) will appear well-calibrated by ECE despite its near-random discrimination.

On calibration, GP models maintain ECE ≤ 0.027 from 425 training observations onward, while classical models are miscalibrated at all scales. XGBoost ECE actually worsens from 0.164 (small) to 0.196 (large): with more data XGBoost makes sharper, more confident probability assignments without an intrinsic calibration mechanism to align those sharper scores with observed frequencies, so the underlying systematic miscalibration becomes more exposed at the larger scale.

6.3.5 Computational Profile

Table 6: Training time scaling across tiers. GP models scale approximately linearly with N; DSVGP-3L is $\sim 3.6\times$ the cost of DSVGP-2L at all scales. All results produced on a MacBook Air M2 CPU (2023).

Model	Time (s, small)	Time (s, medium)	Time (s, large)
Logistic Regression	0.003	0.01	0.03
Random Forest	0.43	1.42	4.85

Model	Time (s, small)	Time (s, medium)	Time (s, large)
XGBoost	0.02	0.08	0.24
SVGP	3.1	40.4	160.4
DSVGP-2L	3.9	46.5	197.0
DSVGP-3L	14.5	149.9	706.5

DSVGP-3L is the most expensive model at 707 seconds, approximately 3.6× the cost of the two-layer variant, yet gives no meaningful improvement over DSVGP-2L on any metric at large scale (ROC-AUC: 0.776 vs 0.777; ECE: 0.021 vs 0.014).

For deployment contexts, the GP training time penalty (197 s for DSVGP-2L vs 0.24 s for XGBoost at large scale) is a batch training cost incurred once per model refresh cycle. In credit-risk production environments where models are retrained monthly or quarterly, this is operationally irrelevant. Inference time for a single prediction is comparable across model families.

6.3.6 Summary

GP and DGP classifiers match XGBoost on discrimination (within 0.003 ROC-AUC, well within test-set variability) while providing 4–22× better native calibration than the uncalibrated classical models, across all training set sizes from 425 observations upward.

SVGP and DSVGP-2L perform similarly on all metrics (ROC-AUC differences of about 0.001), while DSVGP-3L adds ∼3.6× training cost over DSVGP-2L with no measurable gain and fails catastrophically at small scale. Given no improvement from depth, the single-layer SVGP is the more practical choice here, and is easier to justify to regulators than a less interpretable deep architecture.

The GP training time increase (∼820× vs XGBoost) is still under 10 minutes, which is acceptable for infrequent model refreshes (Section 6.3.5).

The lack of DGP improvement over SVGP is consistent with Section 6.2, confirming that the compositional flexibility of DGPs only yields gains on non-stationary target functions rather than the smooth credit-default surfaces typical of financial risk.

7.0 Conclusion

This dissertation set out to answer whether Deep Gaussian Processes improve on single-layer GPs for real-world machine learning tasks, and if so, when and at what

cost.

7.1 Key Findings

DGPs outperform single-layer GPs only on non-stationary, compositional targets. On the Kin8nm robot kinematics benchmark, DSVGP-2L reduced RMSE by 10.7% over SVGP (0.0699 vs 0.0783) and improved NLPD from -1.116 to -1.241. This is consistent with Kin8nm's target being a composition of trigonometric functions of the joint angles, the kind of hierarchical non-linearity where learned intermediate representations add value. On the UCI credit card default benchmark, where the target surface is smoother and more monotonic, DGPs matched but did not exceed single-layer GP performance.

The GP family delivers substantially better native probability calibration than classical models. On the credit card classification task, GP models achieved ECE of 0.010–0.021, compared to 0.087–0.225 for random forest, XGBoost, and logistic regression, a 4–22$\times$ improvement with no post-hoc calibration required. This held across all training set sizes from 425 observations onward. Under Basel IRB, where PD estimates must track observed default frequencies across rating grades, this removes the post-hoc recalibration step XGBoost would need, though full compliance involves much more than calibration metrics.

DSVGP-3L consistently fails to justify its extra compute costs. Across both datasets and all training scales, three-layer DGPs never meaningfully outperformed two-layer variants on any metric. At small training set sizes ($N \leq 500$), DSVGP-3L failed completely with $R^2 = 0.002$ on Kin8nm small scale and ROC-AUC = 0.523 on UCI small scale. They recovered fully by medium scale. At full scale, the extra layer added approximately 3.6–4$\times$ training time and gave negligible gains. DSVGP-2L is therefore the more defensible deep configuration.

The three-layer collapse observed here may be linked to the constant-mean hidden-layer architecture we used. Linear-mean ("skip-connected") variants (Salimbeni and Deisenroth 2017) have evidence showing they may somewhat mitigate this, but were not tested. Three-layer DGPs may also prove to be more beneficial on very non-stationary targets at larger scales, but there was no evidence of this on the datasets tested here.

The right level of model complexity is determined by the structure of the target function, not by theoretical expressiveness alone. DGPs add genuine value when the problem demands it, but without non-stationarity and compositional target structures, single-layer GPs appear preferable.

7.2 Limitations

These findings have several limitations. First, all experiments were conducted on CPU (Apple M2) without GPU acceleration. Absolute training times would be considerably shorter on modern hardware, though relative ratios between models should remain roughly similar. Second, only two datasets were evaluated, and so a broader benchmark across more domains and datasets would be needed to establish any generalisation claims. Third, the DGP architectures tested use fixed-width hidden layers, and neither variable-width architectures nor residual-style DGPs with linear mean functions (Salimbeni and Deisenroth 2017) were explored. Finally, the regulatory compliance claims are illustrative rather than definitive. Real-world PD model validation involves back-testing, portfolio-level stability testing, and model documentation requirements that extend well beyond the metrics evaluated here.

7.3 Future Directions

There are several directions for future work. Stochastic-depth DGPs, analogous to stochastic depth in deep neural networks, where layers are skipped randomly during training, may address the small-data collapse of deep architectures. On the application side, regulatory back-testing of GP-based PD models, and evaluating stability of calibration across economic cycles and portfolio segments, would provide stronger evidence for or against adoption of GPs in practice.

References

Basel III: A Global Regulatory Framework for More Resilient Banks and Banking Systems. 2011. Basel Committee on Banking Supervision, Bank for International Settlements. https://www.bis.org/publ/bcbs189.htm.

Bernardo, José M., and Adrian F. M. Smith. 2000. *Bayesian Theory*. Online-Ausg. Wiley Series in Probability and Statistics. Wiley. https://doi.org/10.1002/9780470316870.

Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer.

Blei, David M., Alp Kucukelbir, and Jon D. McAuliffe. 2017. "Variational Inference: A Review for Statisticians." *Journal of the American Statistical Association* 112 (518): 859–77. https://doi.org/10.1080/01621459.2017.1285773.

Damianou, Andreas C., and Neil D. Lawrence. 2013. "Deep Gaussian Processes." *Proceedings of the Sixteenth International Conference on Artificial Intelligence and Statistics*, March. https://doi.org/10.48550/arXiv.1211.0358.

Dunlop, Matthew M, Mark A Girolami, Andrew M Stuart, and Aretha L Teckentrup. 2018. "How Deep Are Deep Gaussian Processes?" *Journal of Machine Learning Research* 19: 1–46.

Duvenaud, David, Oren Rippel, Ryan P. Adams, and Zoubin Ghahramani. 2014. "Avoiding Pathologies in Very Deep Networks." *Proceedings of the 17th International Conference on Artificial Intelligence and Statistics*, Proceedings of machine learning research, vol. 33: 202–10.

European Banking Authority. 2017. *Guidelines on PD Estimation, LGD Estimation and the Treatment of Defaulted Exposures*. EBA/GL/2017/16. European Banking Authority. https://www.eba.europa.eu/regulation-and-policy/model-validation/guidelines-on-pd-lgd-estimation-and-treatment-of-defaulted-assets.

Gelman, Andrew, John B Carlin, Hal S Stern, David B Dunson, Aki Vehtari, and Donald B Rubin. 2013. *Bayesian Data Analysis*. 3rd ed. 3rd ed.

Hensman, James, Nicolo Fusi, and Neil D. Lawrence. 2013. "Gaussian Processes for Big Data." *Proceedings of the 29th Conference on Uncertainty in Artificial Intelligence*, 282–90.

Huang, Shian-Chang. 2011. "Using Gaussian Process Based Kernel Classifiers for Credit Rating Forecasting." *Expert Systems with Applications* 38 (7): 8607–11. https://doi.org/10.1016/j.eswa.2011.01.064.

International Accounting Standards Board. 2014. *IFRS 9 Financial Instruments*. IFRS Foundation. https://www.ifrs.org/issued-standards/list-of-standards/ifrs-9-financial-instruments/.

Jordan, Michael I., Zoubin Ghahramani, Tommi S. Jaakkola, and Lawrence K. Saul. 1999. "An Introduction to Variational Methods for Graphical Models." *Machine Learning* 37 (2): 183–233. https://doi.org/10.1023/A:1007665907178.

Kendall, Alex, and Yarin Gal. 2017. "What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?" *Advances in Neural Information Processing Systems* 30. https://proceedings.neurips.cc/paper/2017/hash/2650d6089a6d640c5e85b2b88265dc2b-Abstract.html.

Kingma, Diederik P., and Jimmy Ba. 2015. "Adam: A Method for Stochastic Optimization." *Proceedings of the 3rd International Conference on Learning Representations*, January. https://doi.org/10.48550/arXiv.1412.6980.

Kingma, Diederik P., and Max Welling. 2014. "Auto-Encoding Variational Bayes." *Proceedings of the 2nd International Conference on Learning Representations*.

Lehdili, N., P. Oswald, and H. D. Nguyen. 2025. "Performance-Enhancing Market Risk Calculation Through Gaussian Process Regression and Multi-Fidelity Modeling." *Computation* 13 (6). https://doi.org/10.3390/computation13060134.

MacKay, David J. C. 1992. *A Practical Bayesian Framework for Backpropagation Networks*. https://doi.org/10.1162/neco.1992.4.3.448.

MacKay, David JC. 1998. "Introduction to Gaussian Processes." *NATO ASI Series F Computer and Systems Sciences* 168: 133–66. https://www.inference.org.uk/mackay/gpB.ps.gz.

Mahajan, Shubham, Anand Nayyar, Akshay Raina, Samreen J. Singh, Ashutosh Vashishtha, and Amit Kant Pandit. 2022. "A Gaussian Process-Based Approach Toward Credit Risk Modeling Using Stationary Activations." *Concurrency and Computation: Practice and Experience* 34 (5): e6692. https://doi.org/10.1002/cpe.6692.

Munoz-Gonzalez, Luis, Miguel Lazaro-Gredilla, and Anibal R. Figueiras-Vidal. 2014. "Laplace Approximation with Gaussian Processes for Volatility Forecasting." *2014 4th International Workshop on Cognitive Information Processing (CIP)* (Copenhagen, Denmark), May, 1–6. https://doi.org/10.1109/CIP.2014.6844502.

Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. 4th printing. Adaptive Computation and Machine Learning Series. MIT Press.

Neal, Radford M. 1996. *Bayesian Learning for Neural Networks*. Springer New York.

Niculescu-Mizil, Alexandru, and Rich Caruana. 2005. "Predicting Good Probabilities with Supervised Learning." *Proceedings of the 22nd International Conference on Machine Learning*, 625–32. https://doi.org/10.1145/1102351.1102430.

Platt, John C. 1999. "Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods." In *Advances in Large Margin Classifiers*, edited by Alexander J. Smola, Peter Bartlett, Bernhard Sch"olkopf, and Dale Schuurmans. MIT Press.

Quinonero-Candela, Joaquin, and Carl Edward Rasmussen. 2005. "A Unifying View of Sparse Approximate Gaussian Process Regression." *Journal of Machine Learning Research* 6: 1939–59.

Rasmussen, Carl Edward, and Christopher K. I. Williams. 2006. *Gaussian Processes for Machine Learning*. Adaptive Computation and Machine Learning. MIT Press.

Rezende, Danilo Jimenez, Shakir Mohamed, and Daan Wierstra. 2014. "Stochastic Backpropagation and Approximate Inference in Deep Generative Models." *Proceedings of the 31st International Conference on Machine Learning*.

Salimbeni, Hugh, and Marc Deisenroth. 2017. "Doubly Stochastic Variational Inference for Deep Gaussian Processes." *Advances in Neural Information Processing Systems*, November. https://doi.org/10.48550/arXiv.1705.08933.

Schölkopf, Bernhard, and Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. Adaptive Computation and Machine Learning. MIT Press.

Titsias, Michalis. 2009. "Variational Learning of Inducing Variables in Sparse Gaussian Processes." *Proceedings of the 12th International Conference on Artificial Intelligence and Statistics* 5: 567–74. http://proceedings.mlr.press/v5/titsias09a.html.

Williams, Christopher. 1996. "Computing with Infinite Networks." In *Advances in Neural Information Processing Systems*, edited by M. C. Mozer, M. Jordan, and T. Petsche, vol. 9. MIT Press. https://proceedings.neurips.cc/paper_files/paper/1996/file/ae5e3ce40e0404a45ecacaaf05e5f735-Paper.pdf.

Williams, Christopher, and Carl Rasmussen. 1995. "Gaussian Processes for Regression." In *Advances in Neural Information Processing Systems*, edited by D. Touretzky, M. C. Mozer, and M. Hasselmo, vol. 8. MIT Press. https://proceedings.neurips.cc/paper_files/paper/1995/file/7cce53cf90577442771720a370c3c723-Paper.pdf.

Yeh, I-Cheng, and Che-hui Lien. 2009. "The Comparisons of Data Mining Techniques for the Predictive Accuracy of Probability of Default of Credit Card Clients." *Expert Systems with Applications* 36 (2): 2473–80. https://doi.org/10.1016/j.eswa.2008.05.034.

A Appendix

1.1 Design Matrix Construction

With $\mathbf{y} = [y_1, \ldots, y_n]^\top$ and the design matrix $\Phi \in \mathbb{R}^{n \times M}$ where $\Phi_{ij} = \phi_j(\mathbf{x}_i)$, the i-th row of Φ is $\boldsymbol{\phi}(\mathbf{x}_i)^\top$. The vector of predicted means is therefore

$$\begin{bmatrix} \boldsymbol{\phi}(\mathbf{x}_1)^\top \mathbf{w} \\ \vdots \\ \boldsymbol{\phi}(\mathbf{x}_n)^\top \mathbf{w} \end{bmatrix} = \begin{bmatrix} \boldsymbol{\phi}(\mathbf{x}_1)^\top \\ \vdots \\ \boldsymbol{\phi}(\mathbf{x}_n)^\top \end{bmatrix} \mathbf{w} = \Phi \mathbf{w}.$$

1.2 Completing the Square for the Posterior over Weights

Starting from the log-posterior of the Bayesian linear regression model (ignoring terms independent of $\mathbf{w}$):

$$\log p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) \propto -\frac{1}{2\sigma_n^2}(\mathbf{y} - \Phi\mathbf{w})^\top(\mathbf{y} - \Phi\mathbf{w}) - \frac{1}{2}\mathbf{w}^\top \Sigma_p^{-1}\mathbf{w},$$

expanding the quadratic in $\mathbf{y} - \Phi\mathbf{w}$ and collecting terms in $\mathbf{w}$ gives

$$-\frac{1}{2}\mathbf{w}^\top\Big(\sigma_n^{-2}\Phi^\top\Phi + \Sigma_p^{-1}\Big)\mathbf{w} + \sigma_n^{-2}\mathbf{y}^\top\Phi\,\mathbf{w} + \text{const.} \tag{18}$$

We verify that this can be written as $-\frac{1}{2}(\mathbf{w}-\bar{\mathbf{w}})^\top\Sigma_w^{-1}(\mathbf{w}-\bar{\mathbf{w}})$ plus terms independent of $\mathbf{w}$, which identifies the posterior as Gaussian with mean $\bar{\mathbf{w}}$.

Expanding the squared form gives

$$\begin{aligned} -\frac{1}{2}(\mathbf{w} - \bar{\mathbf{w}})^\top\Sigma_w^{-1}(\mathbf{w} - \bar{\mathbf{w}}) &= -\frac{1}{2}(\mathbf{w}^\top - \bar{\mathbf{w}}^\top)\Sigma_w^{-1}(\mathbf{w} - \bar{\mathbf{w}}) \\ &= -\frac{1}{2}\mathbf{w}^\top\Sigma_w^{-1}\mathbf{w} + \frac{1}{2}\mathbf{w}^\top\Sigma_w^{-1}\bar{\mathbf{w}} + \frac{1}{2}\bar{\mathbf{w}}^\top\Sigma_w^{-1}\mathbf{w} - \frac{1}{2}\bar{\mathbf{w}}^\top\Sigma_w^{-1}\bar{\mathbf{w}}. \end{aligned}$$

Since Σ_w^{-1} is symmetric, we have $\bar{\mathbf{w}}^\top\Sigma_w^{-1}\mathbf{w} = \mathbf{w}^\top\Sigma_w^{-1}\bar{\mathbf{w}}$, so the two middle terms combine to give

$$-\frac{1}{2}(\mathbf{w} - \bar{\mathbf{w}})^\top\Sigma_w^{-1}(\mathbf{w} - \bar{\mathbf{w}}) = -\frac{1}{2}\mathbf{w}^\top\Sigma_w^{-1}\mathbf{w} + \mathbf{w}^\top\Sigma_w^{-1}\bar{\mathbf{w}} - \frac{1}{2}\bar{\mathbf{w}}^\top\Sigma_w^{-1}\bar{\mathbf{w}}. \tag{19}$$

Comparing the linear term in $\mathbf{w}$ from Equation 18 with Equation 19, we require

$$\mathbf{w}^\top \Sigma_w^{-1} \bar{\mathbf{w}} = \sigma_n^{-2} \mathbf{y}^\top \Phi \mathbf{w}.$$

Writing the left side as $(\Sigma_w^{-1}\bar{\mathbf{w}})^\top \mathbf{w}$ and the right side as $(\sigma_n^{-2}\Phi^\top \mathbf{y})^\top \mathbf{w}$, and noting this must hold for all $\mathbf{w}$, we get $\Sigma_w^{-1}\bar{\mathbf{w}} = \sigma_n^{-2}\Phi^\top \mathbf{y}$, which gives the posterior mean $\bar{\mathbf{w}} = \sigma_n^{-2}\Sigma_w \Phi^\top \mathbf{y}$ as stated in Equation 2.

1.3 Predictive Mean and Variance via Linear Transformation

Since $f_* = \boldsymbol{\phi}(\mathbf{x}_*)^\top \mathbf{w}$ is a linear function of $\mathbf{w} \mid \mathbf{X}, \mathbf{y} \sim \mathcal{N}(\bar{\mathbf{w}}, \Sigma_w)$, both the mean and variance follow from standard results for linear transformations of Gaussian random variables (Murphy 2012).

Mean:

$$\begin{aligned}
\mathbb{E}[f_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}] &= \mathbb{E}[\boldsymbol{\phi}(\mathbf{x}_*)^\top \mathbf{w} \mid \mathbf{X}, \mathbf{y}] \\
&= \boldsymbol{\phi}(\mathbf{x}_*)^\top \mathbb{E}[\mathbf{w} \mid \mathbf{X}, \mathbf{y}] \\
&= \boldsymbol{\phi}(\mathbf{x}_*)^\top \bar{\mathbf{w}} \\
&= \sigma_n^{-2} \boldsymbol{\phi}(\mathbf{x}_*)^\top \Sigma_w \Phi^\top \mathbf{y}.
\end{aligned}$$

Variance:

$$\begin{aligned}
\mathrm{Var}[f_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}] &= \mathrm{Var}[\boldsymbol{\phi}(\mathbf{x}_*)^\top \mathbf{w} \mid \mathbf{X}, \mathbf{y}] \\
&= \mathbb{E}[(\boldsymbol{\phi}(\mathbf{x}_*)^\top \mathbf{w} - \boldsymbol{\phi}(\mathbf{x}_*)^\top \bar{\mathbf{w}})^2 \mid \mathbf{X}, \mathbf{y}] \\
&= \boldsymbol{\phi}(\mathbf{x}_*)^\top \mathbb{E}[(\mathbf{w} - \bar{\mathbf{w}})(\mathbf{w} - \bar{\mathbf{w}})^\top \mid \mathbf{X}, \mathbf{y}]\, \boldsymbol{\phi}(\mathbf{x}_*) \\
&= \boldsymbol{\phi}(\mathbf{x}_*)^\top \Sigma_w \boldsymbol{\phi}(\mathbf{x}_*).
\end{aligned}$$

1.4 Covariance Blocks of the Joint Distribution

With $\mathbf{y} = \Phi\mathbf{w} + \boldsymbol{\epsilon}$ where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_n^2 \mathbf{I})$ is independent of $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \Sigma_p)$, and $f_* = \boldsymbol{\phi}(\mathbf{x}_*)^\top \mathbf{w}$, the three covariance blocks are computed as follows.

Output covariance:

$$\begin{aligned}
\mathrm{Cov}[\mathbf{y}, \mathbf{y}] &= \mathrm{Cov}[\Phi\mathbf{w} + \boldsymbol{\epsilon}, \Phi\mathbf{w} + \boldsymbol{\epsilon}] \\
&= \Phi\, \mathrm{Cov}[\mathbf{w}] \Phi^\top + \mathrm{Cov}[\boldsymbol{\epsilon}, \boldsymbol{\epsilon}] \\
&= \Phi \Sigma_p \Phi^\top + \sigma_n^2 \mathbf{I}.
\end{aligned}$$

Cross-covariance (noise $\boldsymbol{\epsilon}$ is independent of f_*):

$$\begin{aligned}\operatorname{Cov}[\mathbf{y}, f_*] &= \operatorname{Cov}[\Phi\mathbf{w} + \boldsymbol{\epsilon},\, \boldsymbol{\phi}(\mathbf{x}_*)^\top \mathbf{w}] \\ &= \Phi \operatorname{Cov}[\mathbf{w}]\boldsymbol{\phi}(\mathbf{x}_*) = \Phi\Sigma_p\boldsymbol{\phi}(\mathbf{x}_*).\end{aligned}$$

Test-point variance:

$$\operatorname{Cov}[f_*, f_*] = \boldsymbol{\phi}(\mathbf{x}_*)^\top \operatorname{Cov}[\mathbf{w}]\boldsymbol{\phi}(\mathbf{x}_*) = \boldsymbol{\phi}(\mathbf{x}_*)^\top\Sigma_p\boldsymbol{\phi}(\mathbf{x}_*).$$

1.5 Credit-Risk Pipeline Snippets

The main text refers to implementation details of the empirical experiments. The excerpts below are included so that the claims in Section 6.2 and Section 6.3 can be tied directly to the code used to generate the results.

1.5.1 Multi-Tier Split Protocol

Both datasets use the same tiered evaluation design. A fixed 20% test set is stratified-sampled once from the full dataset; the remaining 80% forms the development pool. Three training-set sizes are drawn from this pool (small, medium, large), with 15% of each tier's development set reserved for validation:

```
# Split off fixed test set (20 % of full data)
X_dev, X_test, y_dev, y_test = train_test_split(
    X_raw, y_raw, test_size=0.2, stratify=y_raw, random_state=SEED,
)

# Within each tier: subsample dev, then split train/val
def run_tier(X_dev, y_dev, X_test, y_test, label, n_dev):
    if n_dev is not None:
        Xd, yd = _subsample(X_dev, y_dev, min(n_dev, len(y_dev)))
    else:
        Xd, yd = X_dev, y_dev

    X_train, X_val, y_train, y_val = train_test_split(
        Xd, yd, test_size=VAL_FRAC, stratify=yd, random_state=SEED,
    )
    # ... preprocess (fit scaler on train only), then run shared
     ↪ pipeline
```

This produces a three-way split (train / validation / test) per tier. The validation set is used for XGBoost early stopping; all other models train on the combined train+validation set. All tiers share the same held-out test set, ensuring comparability across data-efficiency curves.

1.5.2 Pre-Processing Pipeline

The `DataCleaner` class handles collapsing undocumented categorical codes and applying log-transforms to skewed monetary variables. These run before encoding and scaling.

```
@staticmethod
def _fix_education(df: pd.DataFrame) -> pd.DataFrame:
    """Collapse undocumented EDUCATION codes (0, 5, 6) into 'Other'
    ↪ (4)."""
    df[COL_EDUCATION] = df[COL_EDUCATION].replace({0: 4, 5: 4, 6: 4})
    return df

@staticmethod
def _fix_marriage(df: pd.DataFrame) -> pd.DataFrame:
    """Collapse undocumented MARRIAGE code 0 into 'Other' (3)."""
    df[COL_MARRIAGE] = df[COL_MARRIAGE].replace({0: 3})
    return df

@staticmethod
def _apply_log_transforms(df: pd.DataFrame) -> pd.DataFrame:
    """
    Apply log1p transforms to right-skewed monetary columns.
    LIMIT_BAL and PAY_AMT_* (non-negative): log1p(clip(x, lower=0)).
    BILL_* (can be negative): sign(x) * log1p(|x|).
    """
    for col in [limit_col] + pay_amt_cols:
        if col in df.columns:
            df[col] = np.log1p(df[col].clip(lower=0))
    for col in bill_cols:
        if col in df.columns:
            df[col] = np.sign(df[col]) * np.log1p(np.abs(df[col]))
    return df
```

The `DataPreprocessor` class then handles categorical encoding, numeric scaling, and split construction. The scaler is fit on the training set only and applied to

validation and test sets, preventing leakage.

```
@staticmethod
def _encode_categoricals(X: pd.DataFrame) -> pd.DataFrame:
    """SEX → binary; EDUCATION and MARRIAGE → one-hot
    ↪  (drop_first=True)."""
    X = X.copy()
    if "SEX" in X.columns:
        X["SEX"] = X["SEX"].map({1: 0, 2: 1}).astype(int)  #
↪  1=male→0, 2=female→1
    ohe_cols = [c for c in ["EDUCATION", "MARRIAGE"] if c in
↪  X.columns]
    if ohe_cols:
        X = pd.get_dummies(X, columns=ohe_cols, drop_first=True,
↪  dtype=int)
    return X

def fit_transform(self, X: pd.DataFrame) -> pd.DataFrame:
    """Fit encoders/scaler on X (training set) and return transformed
    ↪  copy."""
    X_enc = self._encode_categoricals(X)
    num_cols = [c for c in _NUMERIC if c in X_enc.columns]
    self._numeric_cols = num_cols
    self._scaler = StandardScaler()
    X_enc = X_enc.copy()
    X_enc[num_cols] =
↪  self._scaler.fit_transform(X_enc[num_cols].astype(float))
    return X_enc

def transform(self, X: pd.DataFrame) -> pd.DataFrame:
    """Apply the already-fitted scaler to a new split (no
    ↪  re-fitting)."""
    X_enc = self._encode_categoricals(X)
    X_enc = X_enc.reindex(columns=self._encoded_cols, fill_value=0)
↪  # align dummies
    X_enc = X_enc.copy()
    X_enc[self._numeric_cols] = self._scaler.transform(
        X_enc[self._numeric_cols].astype(float)
    )
    return X_enc
```

The `_NUMERIC` constant covers `LIMIT_BAL`, `AGE`, the six `PAY_*` ordinal repayment-

status codes, six `BILL_*` amounts, and six `PAY_AMT_*` repayment amounts.

1.5.3 DGP Layer Architecture

Each GP layer in the DSVGP implementation is a `DeepGPLayer` subclass. The design choices below are non-standard and directly affect training stability:

```
class _DeepGPLayer(DeepGPLayer):
    def __init__(self, input_dims, output_dims, num_inducing=128,
                 inducing_pts=None):
        # output_dims=None → scalar output (final layer)
        # output_dims=int  → batch of output_dims independent GPs
        ↪  (hidden)
        batch_shape = torch.Size([]) if output_dims is None \
                      else torch.Size([output_dims])

        var_dist = CholeskyVariationalDistribution(
            num_inducing_points=num_inducing,
↪   batch_shape=batch_shape)
        var_strat = VariationalStrategy(
            self, inducing_pts, var_dist,
            ↪  learn_inducing_locations=True)
        super().__init__(var_strat, input_dims, output_dims)

        # Hidden layers use ConstantMean (learnable bias);
        # output layer uses ZeroMean (bias absorbed into variational
        ↪  params)
        MeanClass = ZeroMean if output_dims is None else
↪   ConstantMean
        self.mean_module  = MeanClass(batch_shape=batch_shape)
        self.covar_module = ScaleKernel(
            RBFKernel(batch_shape=batch_shape,
↪   ard_num_dims=input_dims),
            batch_shape=batch_shape,
        )
        # Initialise lengthscale to sqrt(input_dims).
        # Default l=1 gives near-zero kernel values for standardised
        ↪  data
        # in >~5 dimensions because ||delta_x||² ~ input_dims.
        with torch.no_grad():
            self.covar_module.base_kernel.lengthscale =
            ↪  float(input_dims)**0.5
```

Layers are stacked in `_DSVGPInternalModel`. The first hidden layer's inducing points are subsampled from real training data to prevent the first-layer kernel matrix from collapsing before training begins; subsequent hidden layers use random initialisation:

```
class _DSVGPInternalModel(DeepGP):
    def __init__(self, input_dims, n_layers, hidden_dim,
    ↪  num_inducing,
                 X_init=None):
        super().__init__()
        # Seed first-layer inducing points from actual data
        if X_init is not None:
            idx = torch.randperm(X_init.shape[0])[:num_inducing]
            pts = X_init[idx].clone()
            first_inducing = pts.unsqueeze(0).expand(hidden_dim, -1,
↪  -1).clone()
        else:
            first_inducing = None

        self.hidden_layers = torch.nn.ModuleList()
        self.hidden_layers.append(
            _DeepGPLayer(input_dims, hidden_dim, num_inducing,
                         inducing_pts=first_inducing))
        for _ in range(n_layers - 2):   # additional hidden layers
            self.hidden_layers.append(
                _DeepGPLayer(hidden_dim, hidden_dim, num_inducing))
        self.output_layer = _DeepGPLayer(hidden_dim, None,
        ↪  num_inducing)
        self.likelihood = BernoulliLikelihood()

    def forward(self, x):
        h = x
        for layer in self.hidden_layers:
            h = layer(h)
        return self.output_layer(h)
```

For the regression variant, `BernoulliLikelihood` is replaced by `GaussianLikelihood` and the output layer returns a scalar GP with predictive mean and variance.

1.5.4 DSVGP Training Loop

```
optimizer = torch.optim.Adam(self._model.parameters(), lr=self.lr)
mll = DeepApproximateMLL(
    VariationalELBO(
        self._model.likelihood,
        self._model,
        num_data=len(y),
    )
)

train_samples = min(self.num_samples, 10)

for epoch in range(1, self.n_epochs + 1):
    for X_batch, y_batch in _iter_batches(X, y, self.batch_size):
        X_batch = X_batch.to(self.device)
        y_batch = y_batch.to(self.device)

        with gpytorch.settings.num_likelihood_samples(train_samples):
            optimizer.zero_grad()
            output = self._model(X_batch)
            loss   = -mll(output, y_batch)
            loss.backward()
            optimizer.step()
```

This is the doubly stochastic variational setup used for the deep GP models. The first source of stochasticity is mini-batch optimisation; the second is Monte Carlo sampling through the layered latent structure via `num_likelihood_samples`. The deep GP experiments in this dissertation use the DSVI-style training approach of Salimbeni and Deisenroth (2017).

1.5.5 Regression-Specific Extensions

The Kin8nm regression benchmark uses the same GP architectures but with a Gaussian likelihood instead of Bernoulli. The key difference is that regression models return both predictive mean and variance, enabling the negative log predictive density (NLPD) metric.

NLPD Metric:

```
def nlpd(y_true: np.ndarray, y_mean: np.ndarray, y_var: np.ndarray)
↪  -> float:
    """Negative log predictive density under a Gaussian predictive
    ↪  distribution."""
    y_var = np.maximum(y_var, 1e-6)  # numerical floor
    nll = 0.5 * np.log(2 * np.pi * y_var) + 0.5 * (y_true - y_mean)
↪  ** 2 / y_var
    return float(np.mean(nll))
```

NLPD measures how well the model's predictive uncertainty captures the true residual distribution - lower is better. Classical models (Ridge, Random Forest, XGBoost) do not provide predictive variances, so NLPD is reported only for GP-family models.

SVGP Regression Prediction:

The regression SVGP uses `GaussianLikelihood` instead of `BernoulliLikelihood`, returning both mean and variance:

```
@torch.no_grad()
def predict(self, X: pd.DataFrame | np.ndarray,
            chunk_size: int = 512) -> tuple[np.ndarray, np.ndarray]:
    """Return (mean, variance) predictions."""
    self._model.eval()
    self._likelihood.eval()
    X_arr = np.asarray(X, dtype=np.float32)
    all_means, all_vars = [], []
    for start in range(0, len(X_arr), chunk_size):
        X_chunk = torch.tensor(X_arr[start:start +
↪  chunk_size]).to(self.device)
        f_dist = self._model(X_chunk)
        y_dist = self._likelihood(f_dist)  # GaussianLikelihood
        all_means.append(y_dist.mean.cpu().numpy())
        all_vars.append(y_dist.variance.cpu().numpy())
    return np.concatenate(all_means), np.concatenate(all_vars)
```

DSVGP Regression Prediction:

The deep GP regression model aggregates Monte Carlo samples through the layers:

```
@torch.no_grad()
def predict(self, X: pd.DataFrame | np.ndarray,
            chunk_size: int = 256) -> tuple[np.ndarray, np.ndarray]:
    """Return (mean, variance) predictions."""
    self._model.eval()
    X_arr = np.asarray(X, dtype=np.float32)
    all_means, all_vars = [], []
    for start in range(0, len(X_arr), chunk_size):
        X_chunk = torch.tensor(X_arr[start:start +
↪   chunk_size]).to(self.device)
        with
         ↪  gpytorch.settings.num_likelihood_samples(self.num_samples):
            preds = self._model.likelihood(self._model(X_chunk))
            mean = preds.mean.mean(dim=0).cpu().numpy()  # average
↪   over MC samples
            var = preds.variance.mean(dim=0).cpu().numpy()
        all_means.append(mean)
        all_vars.append(var)
    return np.concatenate(all_means), np.concatenate(all_vars)
```

The `num_samples` parameter (default 30 for prediction, 10 for training) controls the Monte Carlo approximation quality; more samples improve accuracy at increased cost.

1.5.6 Hidden Representation Extraction

The hidden-layer analysis in Section 6.2.2 extracts the posterior mean of the first GP layer directly from GPyTorch's variational strategy. For a trained DSVGP-2L regression model loaded as `dgp`:

```
layer0 = dgp.hidden_layers[0]
# variational_strategy(x, prior=False) → posterior q(u | data)
# posterior.mean has shape [hidden_dim, N] = [5, N]; transpose to [N,
↪  5]
with torch.no_grad():
    posterior = layer0.variational_strategy(X_test_tensor,
↪   prior=False)
    hidden_mean = posterior.mean.T.cpu().numpy()  # [N, hidden_dim]
```

The resulting `hidden_mean` matrix is the 5-dimensional posterior mean representation learned by the first GP layer. PCA is then applied to project it to 2D for visualisation, giving the centre panel of Figure 6.

1.5.7 Optimal Threshold Re-computation

After training, metrics are re-computed at each model's F1-optimal threshold rather than the conventional $t = 0.5$. The threshold search scans $[0.01, 0.99]$ at granularity $\Delta t = 0.001$:

```
def find_optimal_threshold(y_true, y_prob, metric="f1", lo=0.01,
↪  hi=0.99,
                           step=0.001):
    thresholds = np.arange(lo, hi + step, step)
    best_t, best_v = 0.5, 0.0
    for t in thresholds:
        y_pred = (y_prob >= t).astype(int)
        if metric == "f1":
            v = f1_score(y_true, y_pred, zero_division=0.0)
        if v > best_v:
            best_v, best_t = v, t
    return best_t, best_v
```

This is particularly important for imbalanced credit-risk datasets, where calibrated probabilities can sit close to the base rate and therefore require substantially lower operating thresholds than $t = 0.5$.

1.6 UCI Credit Card: Supplementary Scaling Figures

The figures below show ROC/precision-recall curves, reliability diagrams, confusion matrices, and threshold tradeoff plots for the UCI credit-card dataset at the **small** ($N_{\text{dev}} = 500$) and **medium** ($N_{\text{dev}} = 5{,}000$) tiers. These supplement the large-tier figures presented in Section 6.3.3.

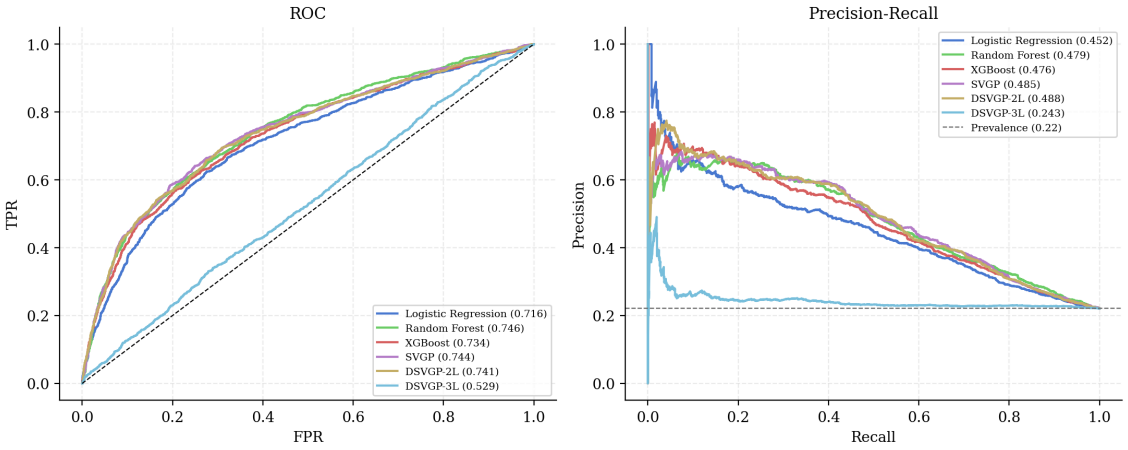


Figure 9: ROC and precision-recall curves - UCI credit card, small tier.

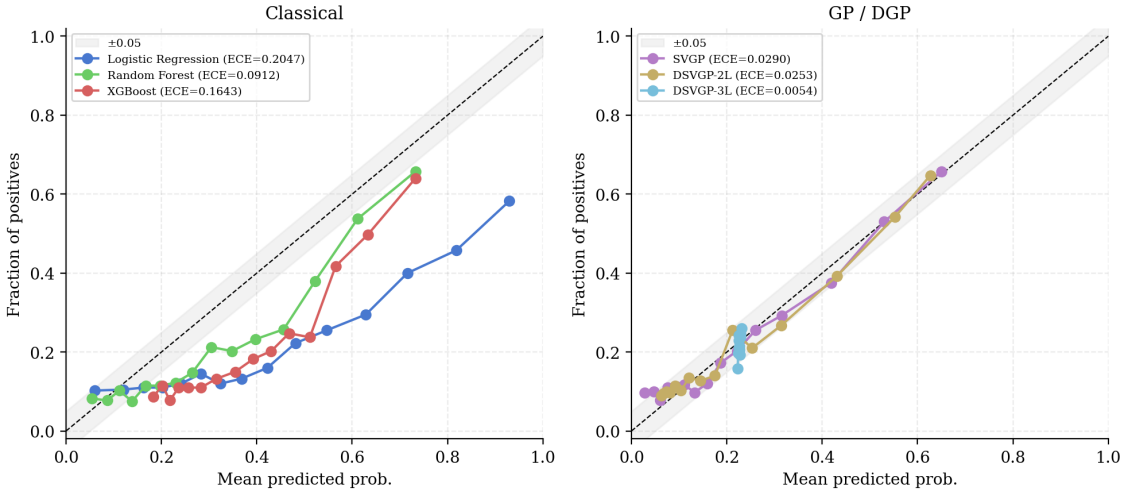


Figure 10: Reliability diagrams - UCI credit card, small tier.

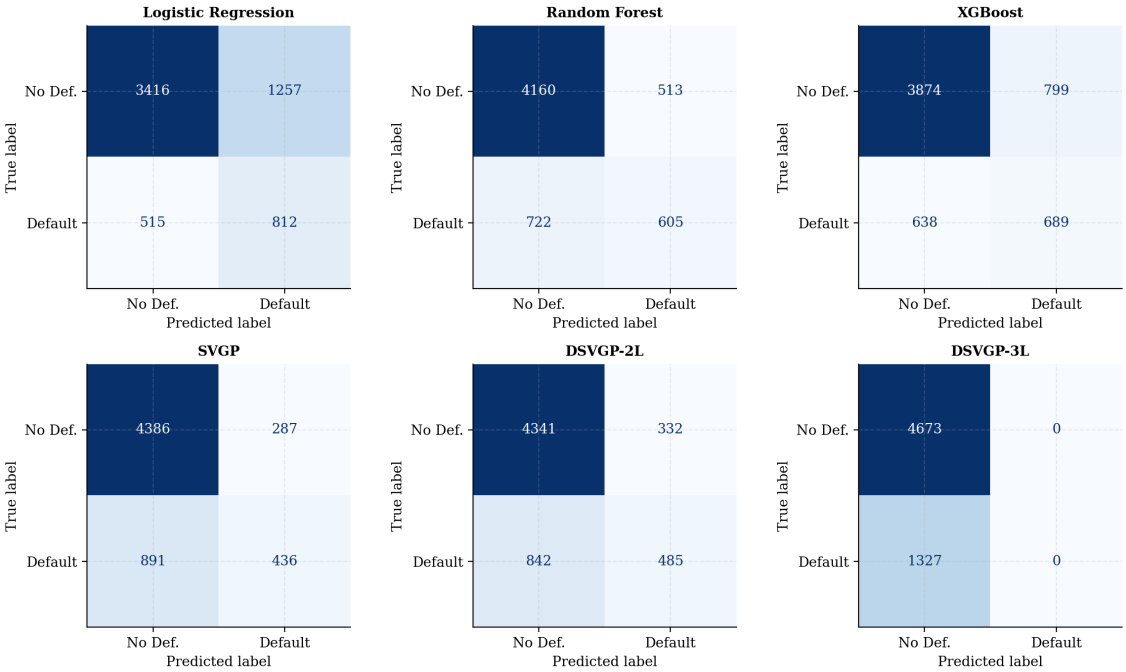


Figure 11: Confusion matrices at optimal threshold - UCI credit card, small tier.

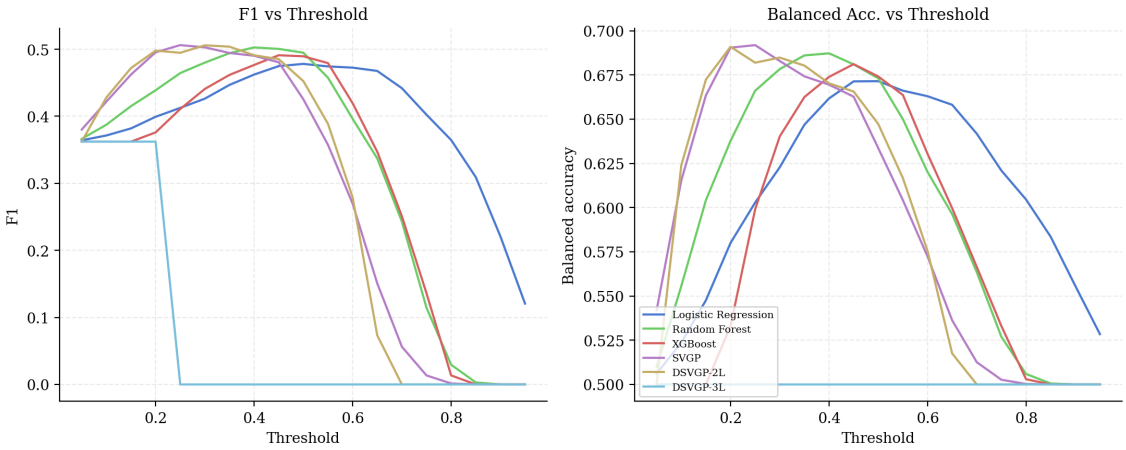


Figure 12: Threshold–F1 tradeoff curves - UCI credit card, small tier.

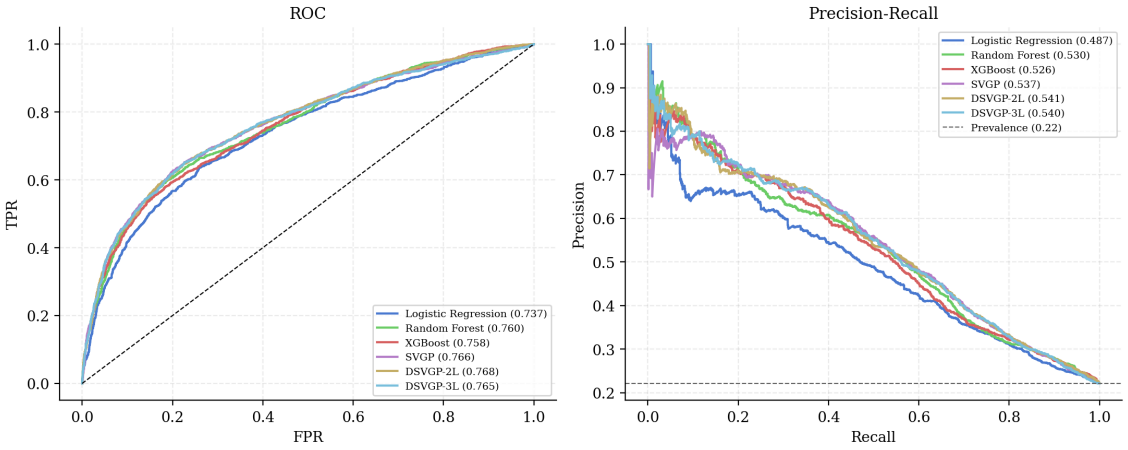


Figure 13: ROC and precision-recall curves - UCI credit card, medium tier.

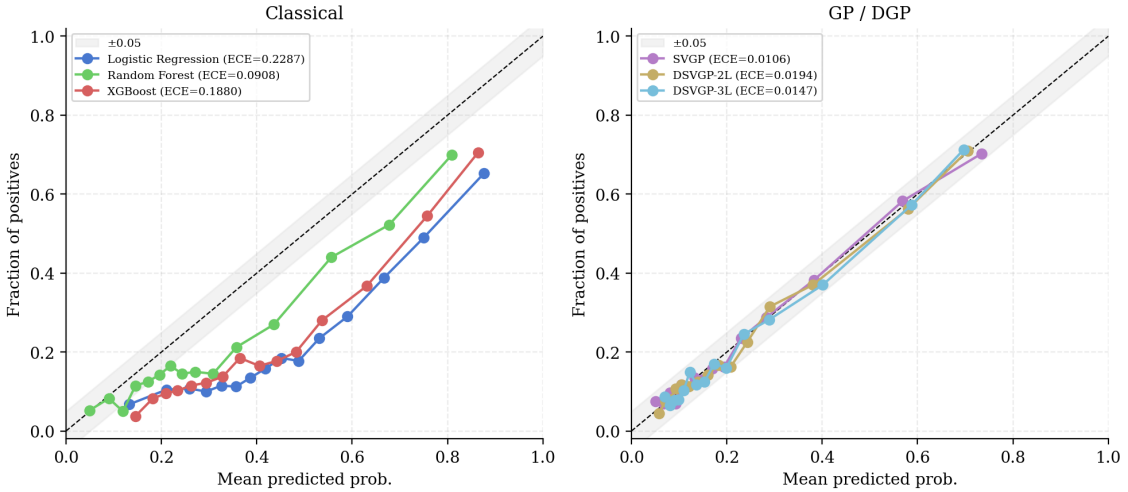


Figure 14: Reliability diagrams - UCI credit card, medium tier.

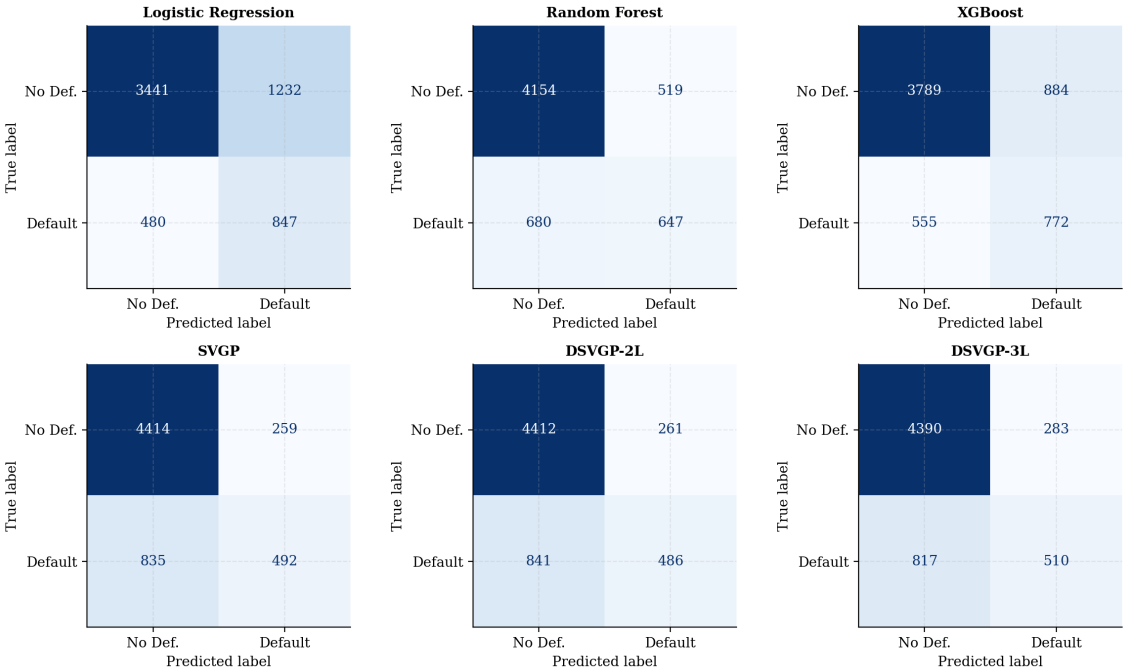


Figure 15: Confusion matrices at optimal threshold - UCI credit card, medium tier.

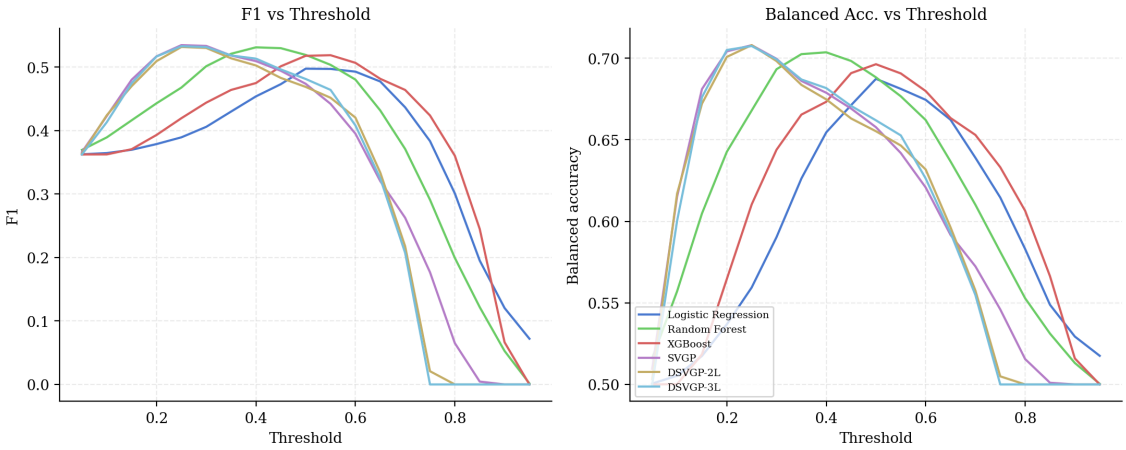


Figure 16: Threshold–F1 tradeoff curves - UCI credit card, medium tier.

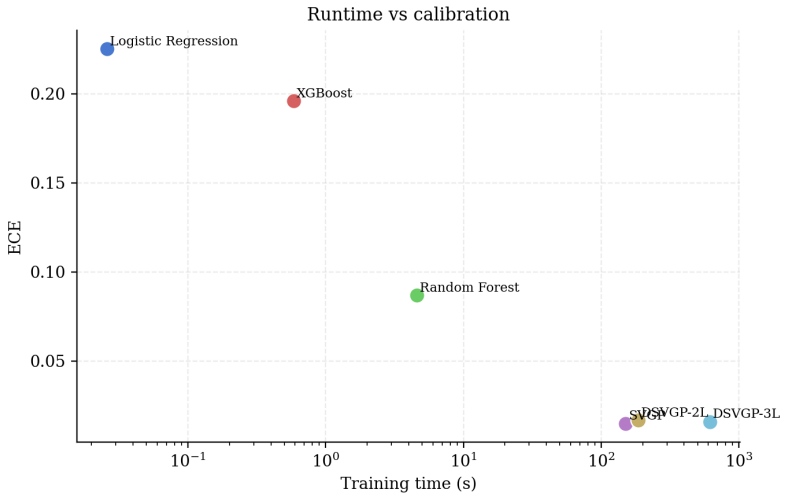


Figure 17: Training time versus ECE for the six models at large scale. GP-family models provide substantially better native calibration at training times two to three orders of magnitude greater than classical models.

1.6.1 Small Tier ($N_{\text{dev}} = 500$)

1.6.2 Medium Tier ($N_{\text{dev}} = 5{,}000$)

1.6.3 Large Tier - Runtime vs Calibration

1.7 Kin8nm: Supplementary Scaling Figures

The figures below show predicted-vs-actual plots, residual distributions, and runtime comparisons for the Kin8nm dataset at the **small** ($N_{\text{dev}} = 500$) and **medium** ($N_{\text{dev}} = 3{,}000$) tiers. These supplement the large-tier figures presented in Section 6.2.2.

1.7.1 Small Tier ($N_{\text{dev}} = 500$)

1.7.2 Medium Tier ($N_{\text{dev}} = 3{,}000$)

1.8 UCI Credit Card: Supplementary Tables

1.8.1 F1-Optimal Threshold Summary - All Models

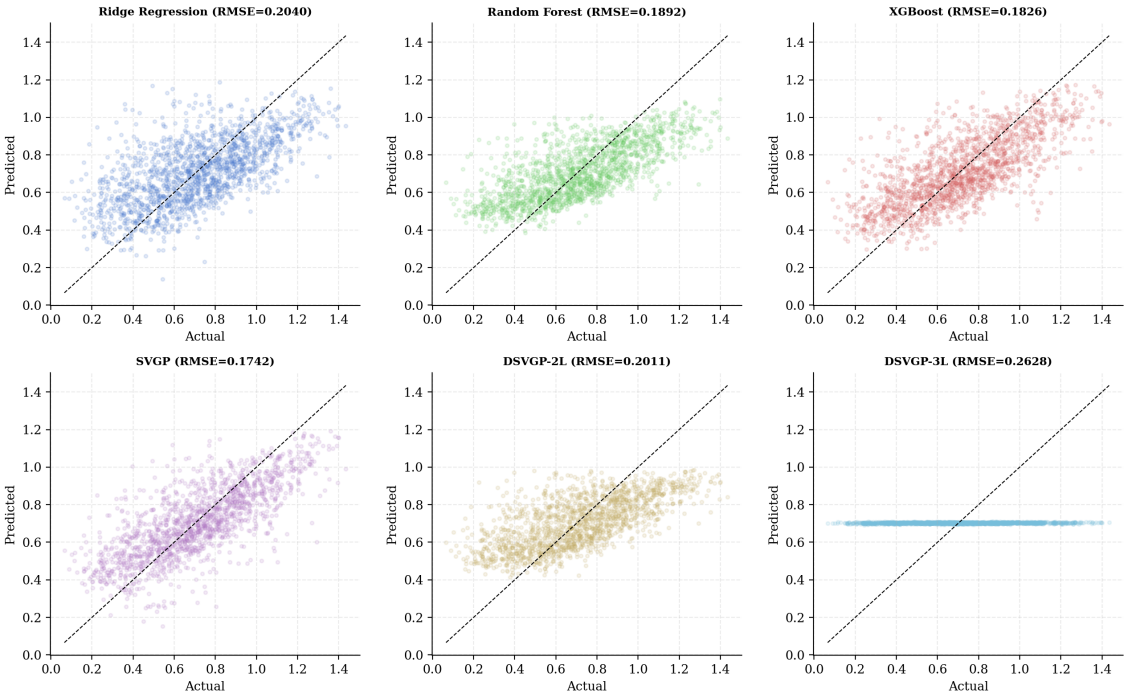


Figure 18: Predicted vs actual values - Kin8nm, small tier.

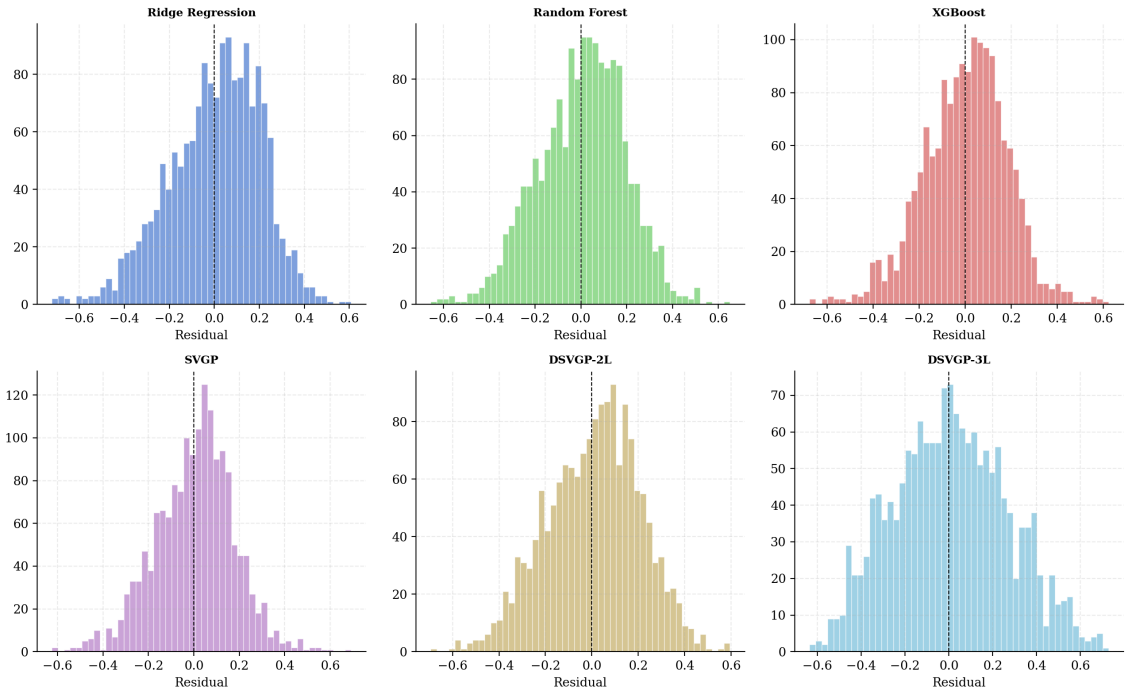


Figure 19: Residual distributions - Kin8nm, small tier.

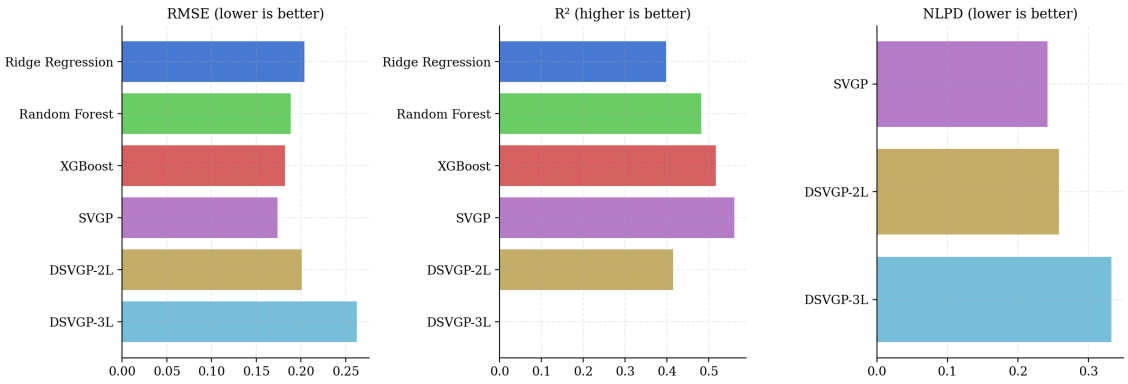


Figure 20: Metrics comparison (RMSE, R^2, NLPD) - Kin8nm, small tier.

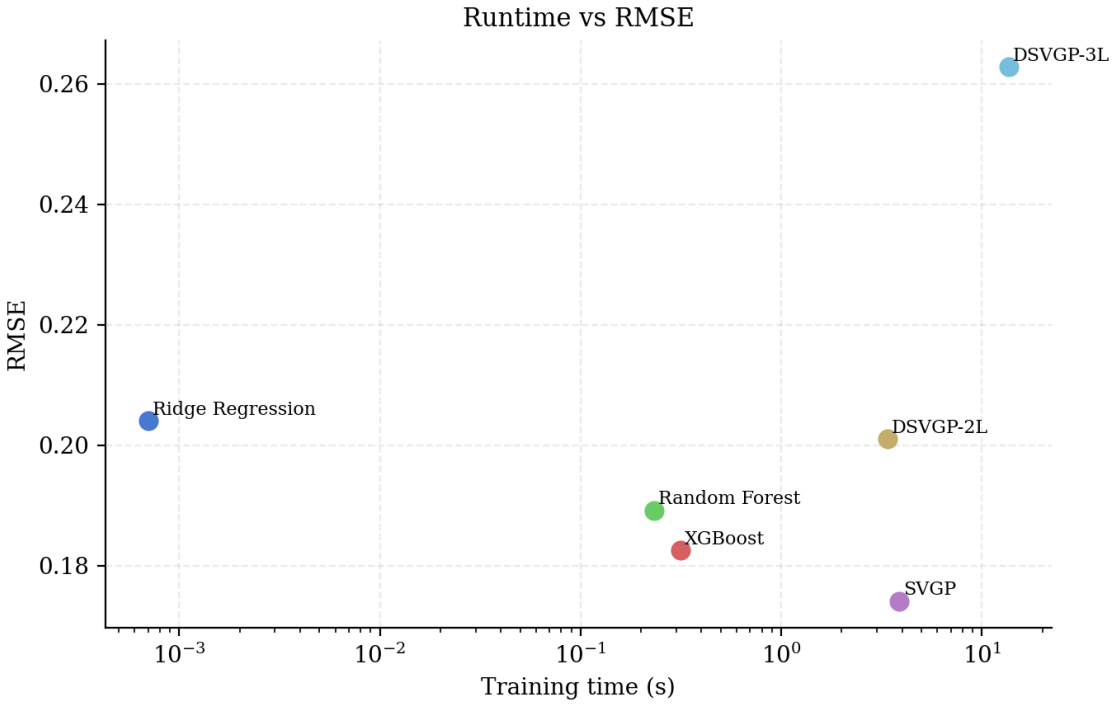


Figure 21: Training runtime - Kin8nm, small tier.

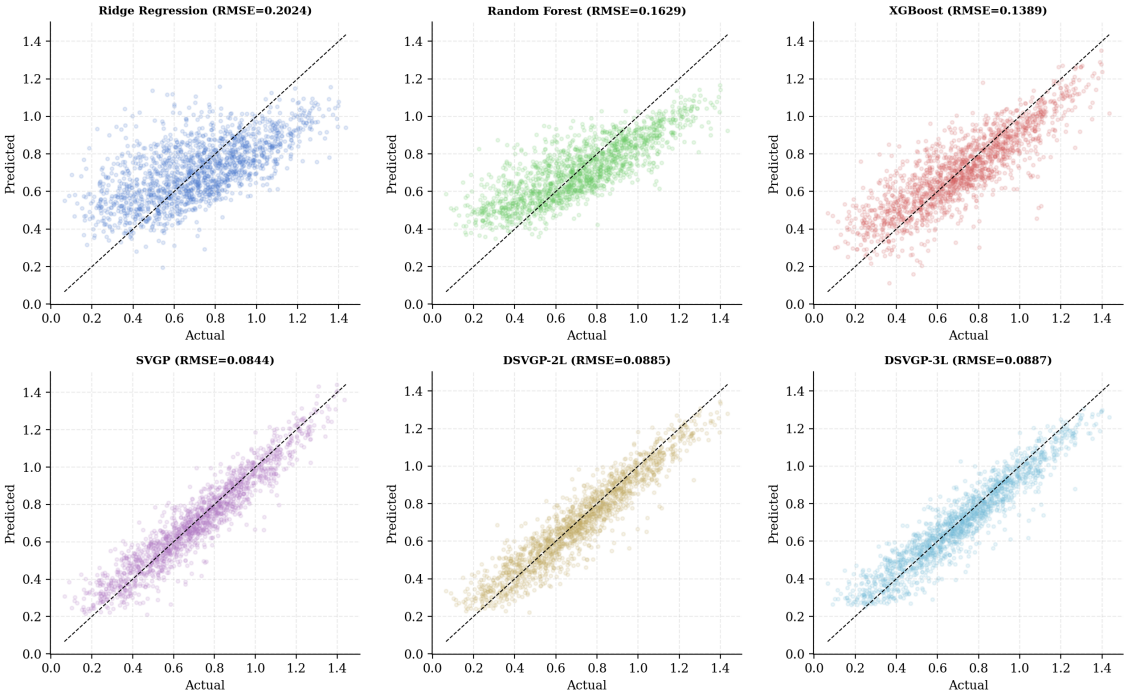


Figure 22: Predicted vs actual values - Kin8nm, medium tier.

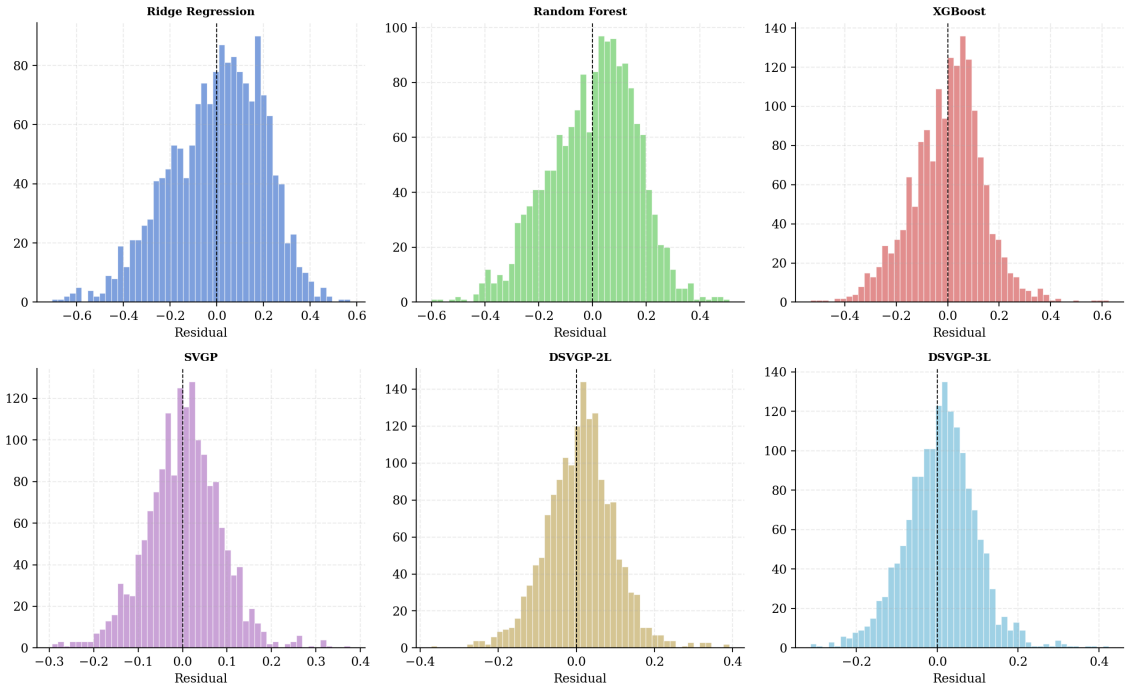


Figure 23: Residual distributions - Kin8nm, medium tier.

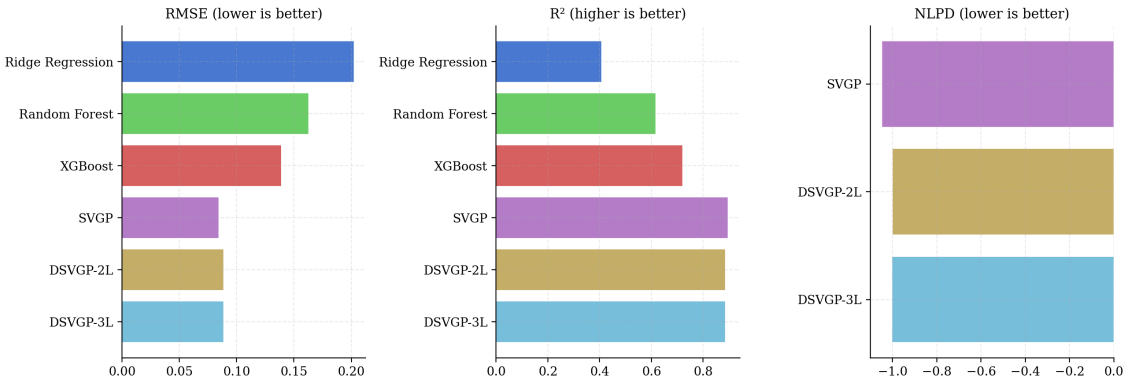


Figure 24: Metrics comparison (RMSE, R^2, NLPD) - Kin8nm, medium tier.

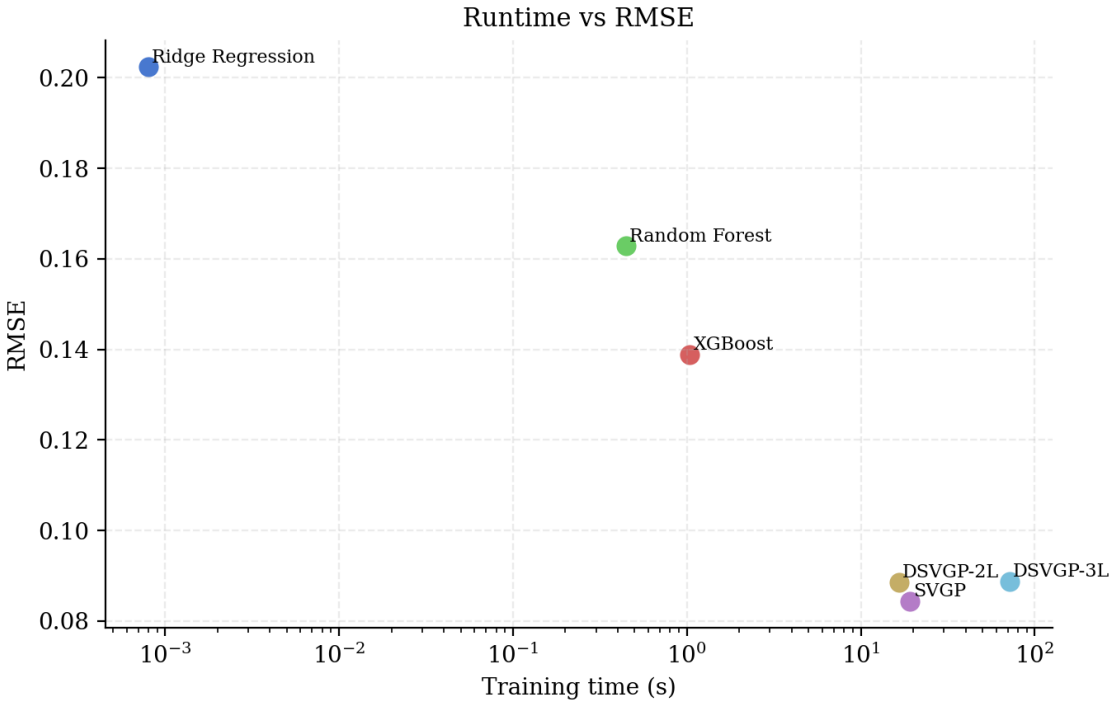


Figure 25: Training runtime - Kin8nm, medium tier.

Table 7: F1-optimal operating thresholds for all six models on the UCI credit-card dataset (large tier). GP models operate at $t^* = 0.30$, approximately 0.20 below the conventional threshold, while classical models range from 0.40–0.55.

Model	t^*	F1 (t^*)	Precision	Recall
Logistic Regression	0.50	0.500	0.421	0.613
Random Forest	0.35	0.532	0.465	0.619
XGBoost	0.50	0.536	0.476	0.613
SVGP	0.30	0.546	0.550	0.543
DSVGP-2L	0.30	0.540	0.533	0.548
DSVGP-3L	0.30	0.544	0.524	0.566

1.8.2 Bootstrap Confidence Intervals

Table 8: Bootstrap 95% confidence intervals for ROC-AUC and ECE on the UCI Credit Card large test set (6,000 observations, 22.1% default rate). The top four models (XGBoost, DSVGP-2L, DSVGP-3L, SVGP) are within 0.003 ROC-AUC, which is smaller than the per-model CI width (~ 0.03); the gap is too small to support that any of these four have better discrimination. ECE intervals are non-overlapping between the GP family (upper bounds ≤ 0.033) and classical models (lower bounds ≥ 0.078), so the calibration advantage is large relative to expected sample variations.

Model	ROC-AUC	95% CI	ECE	95% CI
Logistic Regression	0.739	[0.723, 0.755]	0.225	[0.215, 0.236]
Random Forest	0.769	[0.754, 0.783]	0.087	[0.078, 0.097]
XGBoost	0.779	[0.765, 0.794]	0.196	[0.186, 0.205]
SVGP	0.778	[0.762, 0.792]	0.011	[0.010, 0.023]
DSVGP-2L	0.777	[0.763, 0.792]	0.014	[0.013, 0.027]
DSVGP-3L	0.776	[0.761, 0.791]	0.021	[0.018, 0.033]

1.9 Kin8nm: Supplementary Tables

1.9.1 NLPD Summary - GP Models

Table 9: Negative log predictive density (NLPD) across training tiers. Lower is better. The positive NLPD at small scale for all GP models indicates poorly calibrated predictive uncertainty in the low-data regime - the models' variance estimates are too narrow for their inaccurate means. At sufficient scale, DSVGP-2L achieves the best NLPD.

Model	NLPD (Small)	NLPD (Medium)	NLPD (Large)
SVGP	$+0.243$	-1.048	-1.116
DSVGP-2L	$+0.259$	-1.000	$\mathbf{-1.241}$
DSVGP-3L	$+0.333$	-1.003	-1.158

1.9.2 Runtime Summary - All Models

Table 10: Training time scaling across Kin8nm tiers. DSVGP-3L is approximately $4\times$ the cost of DSVGP-2L at all scales.

Model	Time (s, small)	Time (s, medium)	Time (s, large)
Ridge Regression	0.001	0.001	0.001
Random Forest	0.23	0.45	0.72
XGBoost	0.31	1.04	1.68
SVGP	3.9	19.0	36.6
DSVGP-2L	3.4	16.6	36.2
DSVGP-3L	13.6	71.4	152.4